



MMA League Table (MMA companion)

Computer Science NEA

Thomas Jubb

Contents

Post investigation objective amendments.....	2
Rough Decomposition Flowchart.....	0
Decomposition.....	1
Inputs, outputs and processes.....	7
User Interface	22
Data structure	43
Methods of Access	0
Database design – ERD.....	0
Justification of ERD	0
Data Dictionary.	0
Validation.....	4
Algorithms.....	8

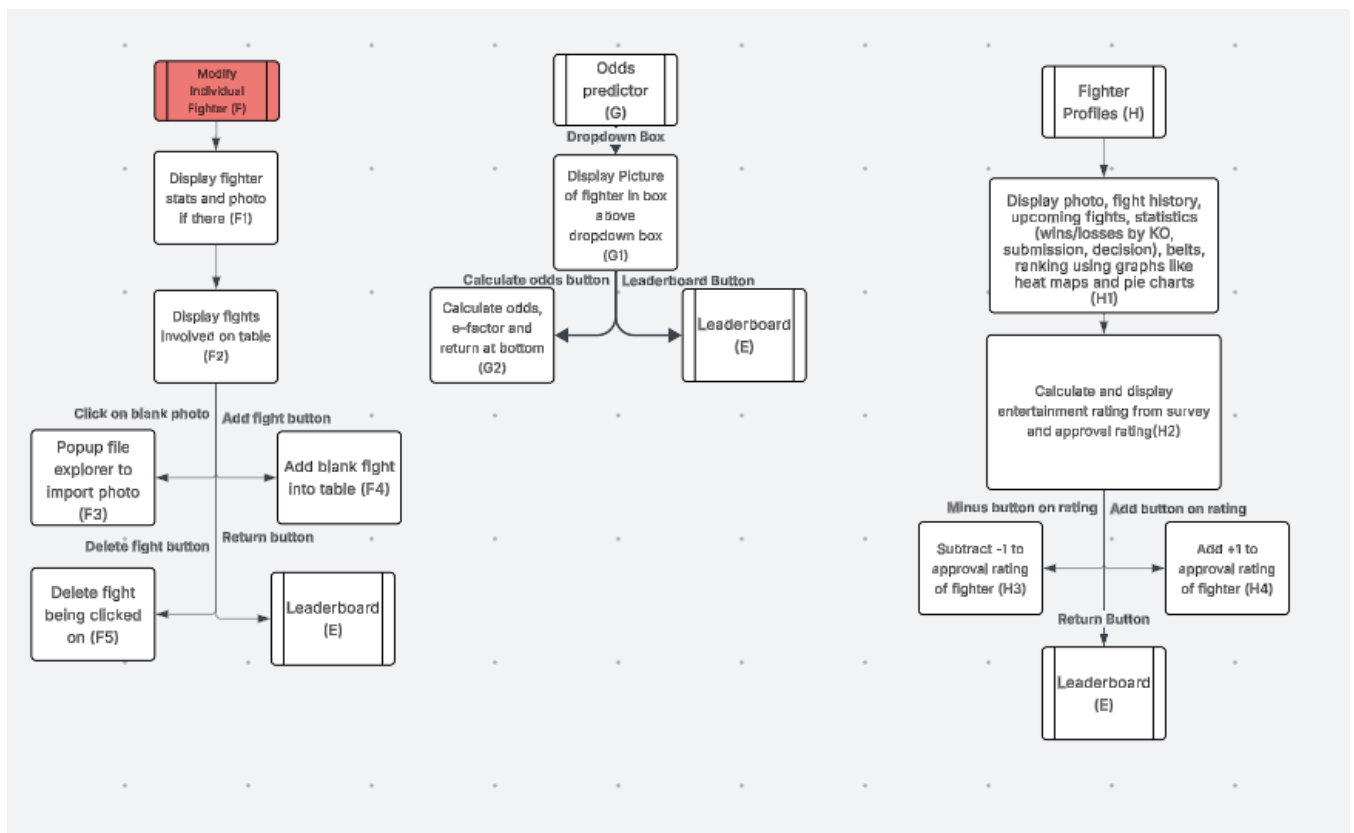
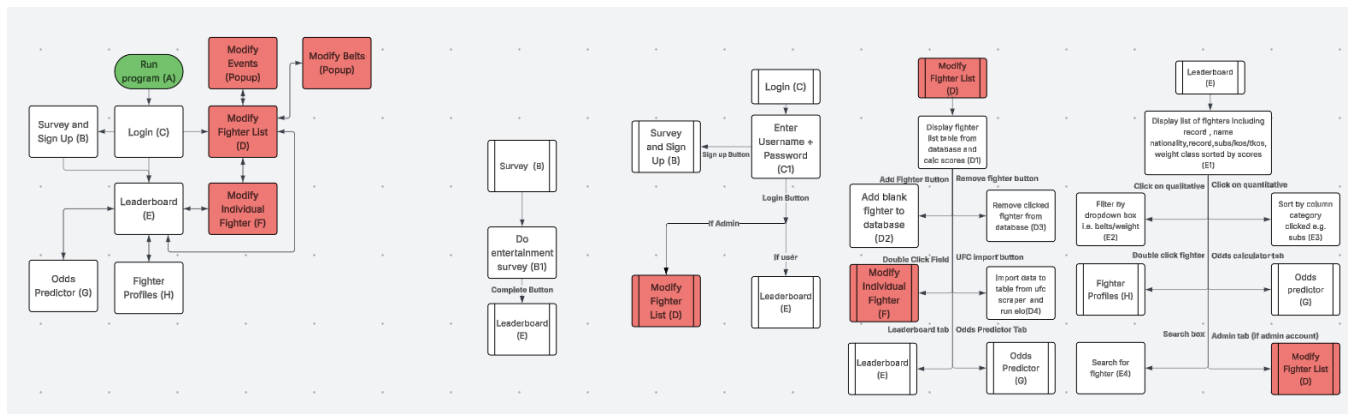
Post investigation objective amendments

Amendment	Objective	Justification
Adding a new objective	The users/admins will be able to navigate between all pages	The user has to be able to navigate between all of the pages so they can actually access everything the program has to offer. I also made this amendment because I want to add clarity on how my navigation works
Adding a new objective	The admin will be able to edit belts and events from a list using a small pop-up	With these two things, there isn't enough data to add for me to justify having a whole page dedicated to them, but it is also quite tedious to allocate space on the existing pages to them, so I'm just going to have these pages pop up when the admin clicks on the fields
Adding a new objective	If there is a general exception or error, it will be displayed using a pop-up.	This is so if a user can refer the error to the admin if it's a problem with servers, and the admin can figure out the problem in a technical sense.
Adding a new objective	The belt will be automatically allocated to the winner of the last declared title fight	This helps ease of use for the admin and decreases the chance of mistakes.
Rewording an objective	Rewording 10 (Upon first use, the system must prompt the user to make a profile so they can answer a short survey about what they find entertaining about an MMA fight. Using this, the system will be able to show how entertaining a fight is going to be to you based on what you like from the survey to just Upon first use, the system must prompt the user to make a profile so they can answer a short survey about what they find entertaining about an MMA fight.	This is because with the second sentence, objective 10 envelops 24 (The system will give each fighter a score according to the fighter's info to determine how entertaining the user will find the fighter.)
Removing an objective	22 (The system will take a survey from the user at the beginning and make a profile that says what they find entertaining and not)	Already covered by 10
Rewording an objective	25 (When the admin makes an update on a table, the program will validate all the data, check for inconsistencies, check for data redundancies, check for statistical anomalies) to just (When the admin makes an update on a table, the program will check for statistical anomalies)	The part I took out is already covered by 14 (The system will validate inputs by rejecting duplicate data and incorrectly formatted records)

Removing an objective	11. Users will also be able to vote on how entertaining a fight is going to be	I don't really see it necessary to include this feature since the probability of users calculating the same fight and then voting enough times for the data to have any value to the user is very slim
Adding an objective	Admins will be able to jump to the leaderboard and back from the modification of the fighter list page	This is so the admins can see what the system will look like with their changes.
Rewording an objective	The system will give each fighter a score according to the fighter's info to determine how entertaining the user will find the fighter. I am going to reword this as: When the user calculates fighter odds, the system will give the fight a score depending on how entertaining they will find it.	I think that the approval rating on the fighter page already is the user-influenced part of the page. I also want to separate the stats and more speculative things, so I think that putting the entertainment score with the odds calculation is a better idea.

Rough Decomposition Flowchart

This diagram I made is a rough vision I made early on in my design to help him home in on what I'm going to do for this project. It shows how the user navigates through the system's processes for each page, which isn't as decomposed (or directly faithful to the objectives) as the subproblems. When it has a box with two lines on either side, it means that the user is navigating to another page (Except on A, because that shows the program's navigation in general). When the box is whole, it means that it's a process. The pages marked in red are Admin-only pages, meaning that only admin accounts can access these pages. (You may need to zoom in a bit, sorry.) The design isn't too faithful to this, but I think it's necessary to show this to see the source of some of my ideas.



Decomposition

Sub-Problem	Justification	Ref
<u>Objective 1: The system will generate ELO scores for every fighter inputted by a system admin using the Glicko 2 model and output a ranked leaderboard (the ELO stuff will be done when the admin updates the table)</u>		
Do a search of all the fights in the database, ordered from oldest to newest	I can calculate the ELO changes chronologically	1.1
Start with the oldest fight, get fighter A's ELO rating and put it into a variable	I can calculate fighter B's ELO despite fighter A changing	1.2
Convert Fighter A and B's ELO and Rating Deviation to the Glicko-2 scale	Storing the fighter's scores in a regular scale makes observation and testing easier and also works better with the odds calculation model.	1.3
Update Rating Deviation based on inactivity	If a fighter is inactive, the system will be a little more sceptical about how they're going to do when they come back.	1.4
Calculate the impact of the fight	If a fighter has a more uncertain rating deviation, i.e. they are inconsistent, then the model will value winning against them less	1.5
Calculate the expected score of the fight	Rating changes will be relative to how you're expected to do, i.e. if you do better than expected, then you will get a better rating change.	1.6
Calculate the variance of the fight	The variance affects the fight rating by figuring out how informative a fight is about the fighter's skill from consistency, past opponents, etc	1.7
Figure out the rating improvement from the fight	The rating improvement figures out how a fighter's rating should change relative to what was expected of them	1.8
Figure out the volatility of ratings	Determines the stability of the rating changes	1.9
Update rating deviation	New update to certainty	1.10
Update main rating	New ELO ratings that reflect the fighter's skill after the fight	1.11
Reconvert RD and score into normal scales from Glicko's scale	Same reason as 2	1.12
Repeat in reverse	This is to handle the changes on the other side	1.13
Repeat for the next oldest fight	Moving on to the next fight	1.14
Display table to User/Admin sorted by external ELO score	Sorting by ranking immediately shows the result of the Glicko algorithm, which is the core component of the hierarchy	1.15
<u>The leaderboard will display the fighter's name, rank number, record, win/loss streak, weight class and methods of winning (number of subs/decisions/kos). (Whenever I refer to displaying the table, this is the general process.)</u>		
Connect to the database	This is to I can interact with the SQL database	2.1
Execute a search query of the fighter list (or if we're doing something else, we do that table), excluding stuff that isn't necessary for the user to see,	This brings up all the stuff we're going to display	2.2

such as photo data and ranking data, ordered by whatever order is needed		
Clear the table we're inserting into	This is to get rid of whatever was there before; if we don't, we may risk mixing old stuff and new stuff.	2.3
Iterate through rows and insert each row into the SQL table	This is to display the table to the user properly	2.4
<u>The table will be sortable by all displayed factors, as well as weight class, nationality, and the belts they've held</u>		
On the user clicking the top bit of their table, sort them by the factor they clicked if it's quantitative. If they click it again, sort it the other way round	Sorting the info by a factor allows the users to see the table by who has the most of whatever factor they want to look at	3.1
On the user clicking a qualitative top bit of the table, do an SQL selection of all the possible qualitative parts of the table, put them all in a dropdown box and then do an SQL selection of all fighters who have that factor.	This allows the users to see all fighters with the qualitative factor they want to see.	3.2
<u>On the user and admin side, you will be able to search for a fighter's name using a Levenshtein (edit based) distance system.</u>		
Find all fighter names after the user presses the search button (if it's blank, deny search)	This brings up all the fighter names so the system can process the closest matches to the search term. The algorithm can deny the search if it's blank because nobody has a blank name.	4.1
Take the first fighter's name and create a 2d matrix list based on the fighter name and the search term	This is so we can go through each variation of the search, so that we can see what cost to the edit amount each variation has	4.2
Go cell to cell, finding the least amount of edits needed for each part of the string	This makes it so that the further we go, the closer to the minimum the edits needed will be	4.3
Take the minimum amount of edits (bottom right if you visualise the list like a matrix) and put that minimum edit distance into a list with the fighter's ID	We are saving the minimum edit distance for later, so we can order it by the smallest edit distance first for the user.	4.4
Repeat these steps for each fighter	This gives us all the Levenshtein distances to the search term	4.5
Quicksort the minimum edit distance list in ascending order	This is because we're going to display the list by the smallest edit distance first to the user	4.6
Redisplay the fighter list to the user/admin, ordered by the minimum edit distance list	The system will output a list ordered by how close to the search term each fighter's name is, which is what the user wants	4.7

<u>5. On the user and admin side, you will be able to click on a fighter's row and then see their profile come up</u>		
On double click, switch page to the fighter page	This is to show the user the page of the fighter.	5.1
<u>6. The fighter profile will feature a photo of the fighter, their past fight history, win streak, upcoming fights, statistics (wins/losses by KO, submission, decision) and belts</u>		
Find the picture of the fighter from the fighter variable and put it onto a widget using a URL	This is to show a photo of the fighter to put a face to the name of the fighters	6.1
Find and display fights where the fighter we're looking at has fought.	This is to show the user the fighter's past opponents to contextualise the skill of a fighter they don't know to other fighters.	6.2
Calculate and display the fighter's win/loss streak	This shows the user what's at stake if the fighter fights, i.e. they lose their win/loss streak	6.3
Calculate total wins/losses/draws and their individual totals (i.e. total KO losses, total sub wins, etc.) and display in a table	This shows the user what to expect in a fighter's fight (i.e. to expect a submission from the fighter if they are a submission artist)	6.4
Calculate the percentage win rate and percentage finish rate for the user	Same reason as the previous sub-problem (to show what to expect)	6.5
Find belts held by the fighter and display them to the user	You want to show what belts someone has to see what's on the line when they fight.	6.6
<u>The fighter profile will also show a pie chart that shows wins to losses and then decomposes to subs/kos/decisions to help users visualise the info.</u>		
Find fight data and calculate total wins and methods of winning, losses and methods of losing	This is so we can put all the data inside the pie chart.	7.1
Create a pie chart for the widget	Like the objective says, this is to allow the user to visualise all the info	7.2
<u>The fighter profile will also show a table that lists the wins that the fighter has against the people who are currently ranked in the top 10 of their division.</u>		
Calculate the 10 th fighter of the fighter we're looking at's division's score.	This is so we can look at all the ones who have a score equal to or greater than the 10 th best fighter (the top 10)	8.1
Find all fighters with ELO equal to or greater than the ELO of the 10 th best fighter in the division, and put that result into a table	This is to show the user the more "notable" fights a fighter has had. Also, the user is way more likely to recognise the higher-ranked fighters.	8.2
<u>The system must take an input of two fighters and give the odds of either of them winning by putting the ELO score they have through the Bradley-Terry model.</u>		
Display all fighters (only ones in the weight of the other fighter if another fighter has been selected in the other box) in a drop-down box, sorted alphabetically	This makes it all compact for the user, but ordered so they don't have to search too much	9.1

When the user clicks calculate, calculate the odds of the fight using a logarithmic Bradley-Terry model	The reason why I used a logarithmic version is that a logarithmic adaptation of the Bradley-Terry model works way with the Glicko ratings we have.	9.2
Display odds in a percentage, depending on who has the dominant rating	This is to display the odds calculation of the fighters to give users an idea of how individual fights will go	9.3
<u>Upon first use, the system must prompt the user to make a profile so they can answer a short survey about what they find entertaining about an MMA fight.</u>		
When the user signs up, the system checks for a duplicate username and then will hash and salt their password and store it along with their username	This is to keep everything as secure as possible. I know the information being stored isn't too sensitive, but people reuse their passwords, so passwords are sensitive information.	10.1
The user will be able to log in again using their username and password.	This is so users can access their approvals and entertainment data again.	10.2
When the user logs in, if everything is valid, the session will be connected to the user by a variable.	This is so the session can figure out what user is using it.	10.3
If the user has an admin account, the session will be an "admin" session (declared by a variable), meaning that they will be able to access admin tabs.	This is so the admin can use their admin tools during their session	10.4
When the user clicks submit on the survey (what they like out of 5 stars), put the 5-star score into another scale and store it in the user's records	This is so we can use bad opinions as a divider and good opinions as a multiplier	10.5
<u>Admins will be able to manually input fighters and fights</u>		
When the admin selects Add Fighter create a blank fighter, the admin can go onto their page and edit	This is so the admins can add more fighters manually to the database	11.1
On the individual fighter editing page, when the admin clicks add fight adds a blank fight to the fighter list that you can edit with dropdown boxes and such	This is to make the admin's life easier, so they don't have to manually put in names/ids	11.2
<u>Admins will also have the option to use a UFC scraper.</u>		
Insert every event since the given date onto the database	From the events, we can link the fights which happen at the events to individual fighters as well.	12.1

Find every fight from every event and add it to the database	This is so we can link all the fights as a baseline, so you don't have any fights without known fighters	12.2
Link every fight to a fighter and build the fighter's profile.	This is so that each fighter has their profile for people to use the system to see UFC stats	12.3
Run Glicko Scores	This is so we can use all the other system stuff with our UFC scraped stats	12.4
<u>The system will validate inputs by rejecting duplicate data and incorrectly formatted records</u>		
Find duplicates in the data, tell the user and make them delete it before all the updates are made	This is so we can make sure all the statistics we do are accurate and that data entry and deletion are way less complicated	13.1
Find incorrectly formatted data, and show the user what's wrong and make them delete it before the updates are made	This is so that the calculations we do on the data work	13.2
Find data of unnecessary length and make the admin delete it before updates are made.	This is to avoid obviously fake data	13.3
Do all necessary presence checks	This is so we can do all the manipulations we want to do	13.4
Make sure all data is the type its supposed to be	This is so all the manipulations we do actually work and don't come up with errors	13.5
<u>Users will be able to vote on fighters and give them a percentage approval rating</u>		
When the plus button is clicked (if not already), store the user's vote in the database	This is to show the user's approval of the fighter	14.1
When the minus button is clicked (if not already), store the user's vote in the database	This is to show the user's disapproval of the fighter	14.2
When the user brings up the fighter's page, view the approval rating as a percentage positive if mostly positive or a percentage negative if mostly negative	This is to show other users' approval of the fighter	14.3
<u>When the user calculates fighter odds, the system will give the fight a score depending on how entertaining the user finds it.</u>		
When the user uses the odds calculation, match each answer the user put on the survey to an attribute of the fighter	This is so we can figure out which attributes the user likes, and what attributes the user doesn't like and how the fighter scales up to it	15.1
Go through each of the attributes relative to the average fighter to figure out what they do more than average (aforementioned), and both fighters are being calculated (this part happens when the user calculates the odds).	This is so we figure out how entertaining the fights will be for the user based on what they like in a fight, which gives them a better idea of how good the fight will be and if it's worth anticipating.	15.2
Calculate the average score of both fighters and output that as the entertainment score	This is to find the midpoint between the two fighters' potential to entertain. This means that if one fighter	15.3

	brings the general entertainment down, the whole rating accounts for that.	
<u>The UI needs to be two-toned and quite saturated to make everything easily readable (be accessible for colour-blind people if possible)</u>		
Avoid red to green where I can if it is crucial to convey data, just use purple to green instead	This is because deuteranopia (red blindness) is the most common form of colour blindness.	
<u>When the admin makes an update on a table, the program will check for statistical anomalies</u>		
Check for impossible fight outcomes (fight lasts 0s)	This is to filter out data that is clearly entered wrong	16.1
Check that every fighter has fought in amounts that are statistically possible (fewer than 12 fights a year)	This is to filter out data that is clearly entered wrong	16.2
<u>The users/admins will be able to navigate between all pages</u>		
When the user navigates to another page or is redirected, specify that all other pages are hidden away from view	This is because the user only wants to see the page they are looking at the time.	17.1
Make sure that all page-specific processes don't happen when the user is away unless specified otherwise	This is so the user doesn't accidentally make things happen that they don't want to happen.	17.2
<u>The admin will be able to edit belts and events from a list using a small pop-up</u>		
When the user clicks a belt button on the edit fighter list page, a list of belts automatically comes up with each holder and division shown (auto-filled from ID)	This is so the admin can edit the belts of the promotion they're working with. Belts are very important because they show who is the champion in an MMA championship.	18.1
The admin can add new belts	This is if a new belt is introduced or the data is incomplete, meaning that you must add titles manually. It could also be for non-divisional belts like the BMF belt in the UFC.	18.2
The admin can delete belts	This is for if a division is discontinued	18.3
If a belt is deleted, the title fight status of the belt's fights will be turned to retired	This is to avoid it messing with the automatic allocation feature I'm about to bring up.	18.4
The admin can edit the holder of the belt	This is for a non-title fight belt change, e.g. a vacation of the belt	18.5
<u>If there is a general exception or error, it will be displayed using a pop-up.</u>		
Open a pop-up window	This is so that the error is brought to the absolute forefront of the UI	19.1
Declare the error in the pop-up window	This is so that the user can see the error that is affecting their program and pass it on to an admin if they are not technically proficient enough	19.2
<u>The belt will be automatically allocated to the winner of the last declared title fight</u>		
Figure out how many weight classes there are	This is so we can figure out how many belts we need	20.1

For each weight class, find the last declared title fight	This is so we can find who won the fight	20.2
Set the belt of the weight class to belong to the last winner of the weight class	This is so we can give the belt to the person who won the last title fight	20.3
<u>Admins will be able to jump to the leaderboard and back from the modification of the fighter list page</u>		
If the user is an admin, when they click the View as User button redirected to the leaderboard	This is so the admin can see what their system looks like in action without having to make a whole user account	21.1
If the user is an admin, display an admin icon on the sidebar	This is so the admin can go back to editing the tables	21.2
<u>The system will make sure that the admin has validated all the data before they exit. (This isn't an objective because it fits in too well with the other objectives, but I want to separate this aspect to emphasise how it works)</u>		
When the admin makes an edit to the data, toggle in the system that they have.	This is so if the admin is just looking at the data, they don't have to submit and validate everything	22.1
If the admin has made an edit, when they exit, submit the data and run algorithms as usual	This is so they can't just shut down the program and mess up their data.	22.2

I have appended some of the objectives from the sub-problems, despite them still being valid. For 16(Rankings need to update when you enter fight results automatically), it is covered through other sub-problems. For 17,18,20,21, they are UI related things that I don't really have to break down.

Inputs, outputs and processes.

Sub Problem	Input	Process	Output	Ref
Do a search of all the fights in the database, ordered from oldest to newest	Each fight in Fighter Fights is referenced to a fight and its fighters Admin clicks the Submit button	SQL search of all fights, ordered by oldest to newest	The fight table, ordered from oldest to newest.	1.1
Start with the oldest fight, get fighter A's ELO	Elo of fighter A	Store ELO into a variable	ELO in variable	1.2

rating and put it into a variable				
Convert Fighter A and B's ELO and Rating Deviation to the Glicko-2 scale	Fighter A and B ELO rating Fighter A and B rating deviation	$g = \frac{r - 1500}{173.7178}$ $gRD = \frac{RD}{173.7178}$ g = Internal Glicko rating r = External ELO rating gRD = Glicko rating deviation RD = external rating deviation If fighters don't have an ELO set it to 1500, If they don't have an RD set it to 350, If they don't have volatility set it to 0.06	Internal Glicko rating Glicko Rating Deviation	1.3
Update Rating Deviation based on inactivity	Old rating deviation Volatility Months since last fight	$rdNEW = \sqrt{rdOLD^2 + \sigma^2 \cdot t}$ rdNEW New Rating deviation rdOLD = Old rating deviation sigma = volatility (updated later, but we use the old fighter volatility) t = months since last fight	New rating deviation	1.4
Calculate the impact of the fight	Opponent's rating deviation	$i_b = \frac{1}{\sqrt{1 + \frac{3q^2\phi_b^2}{\pi^2}}}$ Phi = opponent's rating deviation on internal scale Q = ln(10)/400 (a constant used in Glicko-2 formulas)	Impact factor of fight	1.5
Calculate the expected score of the fight	Impact factor Ratings of each fighter	$E = \frac{1}{1 + e^{-i_b(r_a - r_b)}}$ i = impact factor from fighter b r = rating of fighter a/b internally E = Expected score	Expected score as fighter a vs fighter b	1.6
Calculate the variance of the fight	Impact factor Expected score as fighter a vs fighter b	$v = \left(\sum_b i_b^2 E_b (1 - E_b) \right)^{-1}$ V = Variance	Variance of opponent	1.7

	Number of fights b has (we're going to keep this at 1)	B=1 (I'm keeping it here to make the comparison cleaner, as I have switched around some letters) I= Impact factor E = Expected score		
Figure out the rating improvement from the fight	Variance Expected Score Impact factor Result from fighterfights	$ratingChange = v \sum_b i_b (s_b - E_b)$ V = variance S = Result of fight (1=win, 0.5=draw/NC 0=Loss) E = Expected result	Amount by which the fighter's rating should change	1.8
Figure out the volatility of ratings	V= Variance Sigma = Old volatility Delta = Rating change Phi = Rating Deviation	$f(x) = \frac{e^x(\Delta^2 - \phi^2 - v - e^x)}{2(\phi^2 + v + e^x)^2} - \frac{x - \ln(\sigma^2)}{r^2} = 0$ Sigma= volatility r = constant controlling volatility change (around 0.5) f(x)= this function's roots should give the new volatility Solve for x = ln(sigma) in this equation using a binary search $x = \ln(\sigma^2)$ Then the new volatility = e^(x/2)	New volatility	1.9
Update rating deviation	Old rating deviation Volatility	$\phi^* = \sqrt{\phi^2 + \sigma^2}$ Phi* = Intermediate rating deviation Sigma = volatility $\phi' = \frac{1}{\sqrt{\frac{1}{\phi^{*2}} + \frac{1}{v}}}$ Phi' = new rating deviation V = Variance	New rating deviation	1.10
Update main rating	Old rating Impact factor Outcome loss = 0 draw/nc=0.5 win=1 Expected outcome Rating Deviation	$r' = r + \phi^2 \sum_b i_b (s_b - E_b)$ r = old rating r' = new rating phi = rating deviation i = impact factor	New rating	1.11

		s = outcome e = expected outcome		
Reconvert RD into ELO from Glicko's scale	Internal rating Internal rating deviation	$e = 173.7178i + 1500$ e = external rating i = internal rating $RD = 173.7178\phi'$ RD = external rating deviation Phi = internal rating deviation	External rating External rating deviation	1.12
Repeat in reverse	Stats from the other fighter Variables saved from before	Whole process here, but flipped	New stats updates	1.13
Repeat for the next oldest fight	Stats from the next fighters	Whole process, but with the fighter from the next fight	New stats updates	1.14
Display fighter stats to User/Admin sorted by ELO	Fighter table excluding rating, volatility, and deviation if user	SQL select function for all relevant info, ordered by highest ELO score first, and push to the relevant table	Table viewable to admin/users	1.15
Connect to the database	Server Username	Establish an SQL connection to the server with the parameters we have in the code	Connection to SQL Server	2.1
Execute a search query of the fighter list (or if we're doing something else, we do that table), excluding stuff that isn't necessary for the user to see, such as photo data and ranking data, ordered by whatever order is needed	Query to sort by/ filter	Do an SQL search from the fighter list by the filters (if any), ordered by the quality you want to order by (if there is no order, order by ELO by default)	SQL table to queries and filters	2.2
Clear the table we're inserting into	The table we are inserting into	Iterate through each row and clear each row	A clear table ready to have the data put into it	2.3
Iterate through rows and insert each row into the SQL table	The search result from the table	Iterate through each row of the SQL table and add its result to the table	A table with the desired results inside it	2.4
On the user clicking the top bit of their table, sort them by the factor they clicked if it's	User clicks on one of the titles for the columns of the leaderboard table	Do an SQL search for all the qualitative factors you can get from the column the user has pressed, and put those factors into a drop-down box that comes up. When	Table sorted by whatever qualitative factor	3.1

quantitative. If they click it again, sort it the other way round		the user selects something from that drop-down box, sort the SQL table by what has that quality.		
On the user clicking a qualitative top bit of the table, do an SQL selection of all the possible qualitative parts of the table, put them all in a dropdown box and then do an SQL selection of all fighters who have that factor.	User clicks the qualitative factor on the leaderboard table	If they haven't clicked it yet, sort the table by the highest quantity to lowest. If they have, sort from lowest quantity to highest	Table sorted by whatever quantitative factor	3.2
Find all fighter names after the user presses the search button (if it's blank, deny search)	User clicks the search button	If blank, deny search, and if not blank, search for all fighter names in the database.	List of fighter names	4.1
Take the first fighter's name and create a 2d matrix list based on the fighter name and the search term	List of 2 fighter names	Separate the search term and the first fighter's name. Make a 2d matrix where the rows are the first term you just brought up from the list, and the columns are your search term	A matrix, we can make a Levenshtein distance search	4.2
Go cell to cell, finding the least amount of edits needed for each part of the string	Matrix of two words	Go through each cell and pick the cheapest option from deletion, insertion and substitution.	Levenshtein distance "matrix"	4.3
Take the minimum amount of edits (bottom right if you visualise the list like a matrix) and put that minimum edit distance into a list with the fighter's ID	Levenshtein matrix	The bottom right number on the matrix tells you the minimum edits to get to the fighter name from your search term	Minimum edit distance with the fighter ID in the list	4.4
Repeat these steps for each fighter	Each fighter's name and the search term	Repeat the Levenshtein process for each of these fighters	A list with each fighter and their edit distance to the search term	4.5
Quicksort the minimum edit distance list in ascending order	List, with edit distances	Quicksort	List sorted by the smallest edit distance first	4.6

Redisplay the fighter list to the user/admin, ordered by the minimum edit distance list	Sorted list	Clear the table, iterate, and insert a new sorted table	Sorted table	4.7
On double click, switch page to the fighter page	Double-click on the fighter's record	Put the fighter page to the front with the fighter ID to display the fighter you just clicked.	Fighter page with the fighter you just clicked as the subject	5.1
Find the picture of the fighter from the fighter variable and put it onto a widget	Fighter ID in a variable	Search the fighters table of the database for the photo URL from the fighter ID, request the image from the URL, load it into a pixmap and display it on a label.	The fighter page has a photo of the fighter on it	6.1
Find and display fights where the fighter we're looking at has fought.	Fighter ID from a variable	Search the fights table of the database for all the fights where the fighter we're looking at has fought. Output the data we get from this to an array. Clear the fights table on the fighter page and iterate through the fights table, and insert everything.	Table of fights	6.2
Calculate and display the fighter's win/loss streak	Query results of 6.2 Fighter ID Result from fighterfights table	Go to the most recent fight. If it's a win, add one to a win streak variable. If it's a loss, take one off the win streak variable. Keep going through the fights until the fighter has the opposite value (i.e. a loss after 2 wins would be a streak of +2). Edit a text label with the number, make it red if it's a loss streak, make it green if it's a win streak	Win/loss streak text label	6.3
Calculate total wins/losses/draws and their individual totals (i.e. total ko losses, total sub wins, etc.) and display in a table	Query results of 6.2	Total the number of fights in the table where the fighter in question lost Total the number of fights in the table where the fighter in question won Total the number of fights in the table where the fighter in question got a draw/NC Total the number of fights in the table where the fighter in question had a knockout win/loss Total the number of fights in the table where the fighter in question had a submission win/loss	Total wins/losses/draws with each method sectioned off under it, displayed in 3 labels, 2 of them having 3 sublabels (draws don't have a method)	6.4

		Total the number of fights in the table where the fighter in question had a decision win/loss		
Calculate the percentage win rate and percentage finish rate for the user	Query results from 6.2	Total up the number of fights that are in the fight table, Find the number of wins that are in the fight table Find the number of KO and Submission wins in the fight table (add them up) Do $(100/\text{fights}) * \text{Wins}$ And $(100/\text{wins}) * (\text{KOs} + \text{Subs})$	Finish and win rate are displayed in labels to the user	6.5
Find belts held by the fighter and display them to the user	Belts Holder ID	SQL SEARCH for belts where the holder ID matches the fighter ID	Display belts in list in a text widget to user. If the belts have an image associated with them display that	6.6
Find fight data and calculate total wins and methods of winning, losses and methods of losing	Results of query from 6.2	Total the number of fights in the table where the fighter in question lost Total the number of fights in the table where the fighter in question won Total the number of fights in the table where the fighter in question got a draw/NC Total the number of fights in the table where the fighter in question had a knockout win/loss Total the number of fights in the table where the fighter in question had a submission win/loss Total the number of fights in the table where the fighter in question had a decision win/loss Put wins and methods inside of a list with the structure [wins, submissions, decisions, knockouts, draws/NCs] Do a list of losses as well with the structure of [losses, submissions, decisions, knockouts, draws/NCs]	Data from fights ready to display in an array	7.1
Create a pie chart for the widget	Data on wins	Use the plotting library to display a chart on a widget	Pie chart of wins and losses with the methods of victory/defeat as segments inside the actual parts	7.2
Calculate the fighter of the fighter we're looking at's division's score.	Fighter table (weight class, fighterID, ELO)	SELECT fighters from the fighter table where the fighter's weight class is the same as the fighter	ELO of the 10th-ranked fighter, so we can find notable fights for the fighter's profile	8.1

		we're looking at. Get the ELO of the 10th-ranked fighter.		
Find all fighters with ELO equal to or greater than the ELO of the 10 th best fighter in the division, and put that result into a table	ELO of the 10th-ranked fighter Fights Table Results of 8.1 query	Get fighters from the fighter table where their ELO is greater than or equal to the 10th-ranked fighter and where they have fought the fighter we're looking at (Matches one of the fighter IDs in the fights table where the other matches the ID of the fighter we're looking at). Clear the notable fights list and insert the fighter data we just got.	List of high ranked fighters the fighter we're looking at has gone up against	8.2
Display all fighters (only ones in the weight of the other fighter if another fighter has been selected in the other box) in a drop-down box, sorted alphabetically	Fighter table (weight class, name)	SELECT all fighters to be displayed in the dropdown box. If another fighter has been selected in the other box, SELECT all fighters that are the same weight as the other fighter.	Dropdown boxes to select all the fighters	9.1
When the user clicks calculate, calculate the odds of the fight using a logarithmic Bradley-Terry model	The ELOs of the fighters we've just selected	$p(a \text{ beats } b) = \frac{e^a}{e^a + e^b}$ a=Fighter a rating b=Fighter b rating	Odds of fighter A beating fighter B	9.2
Display odds in a percentage, depending on who has the dominant rating	Odds of Fighter A beating Fighter B	If fighter A is the favourite (i.e. the number is over 0.5 don't change the odds (just multiply it by 100 to make it a percentage) If fighter B is the favourite (under 0.5), then invert the odds (1-odds) and multiply by 100	Display odds and display favourite (e.g. Fighter B favourite at 58%)	9.3
When the user signs up, the system checks for a duplicate username and will hash and salt their password to store it along with their username	New Username New Password Sign-up button pressed	Check for a duplicate username. Reject if there is one When the sign-up button is pressed, generate a random salt (I am going to use PBKDF2-HMAC, so I don't need a salting algorithm) Derive a hash with the salt and the password (which is converted to bits) using a SHA-256 algorithm. Insert a new row into the users table with the username and the hashed password	New profile in the users table If it's a duplicate username displays an error Redirect to Survey	10.1
The user will be able to log in again using their	Username Password Log in button pressed	SELECT from the users table where the username matches the one entered	Login user is it's a match Redirect to leaderboard If not matched, display an error	10.2

username and password.		Separate the hash and salt of the password Recompute the hash for the entered password and then compare it to the one we have.		
When the user logs in, if everything is valid, the session will be connected to the user by a variable.	Successful Login UserID	Set the session Token variable to the UserID	Display the UI relevant to the user with their entertainment scores and approvals	10.3
If the user has an admin account, the session will be an "admin" session (declared by a variable), meaning that they will be able to access admin tabs.	Successful Login UserID	SELECT the isAdmin field from the users table for the user we just logged in as, to see if they're an admin or not. If they set the variable isAdmin to true, which will allow them to see all of the admin pages.	Display the Admin part of the UI if they're an admin. Automatically redirect to Edit Fighter List If they're a user, redirect to Leaderboard	10.4
When the user clicks submit on the survey (what they like out of 5 stars), put the 5-star score into another scale and store it in the user's records	Survey responses Submit button UserID	Turn the survey into a scale of 1-5, going up in increments of 1 for each star. Update the user table with these results where the userID matches.	Redirect to Leaderboard	10.5
When the admin selects Add Fighter create a blank fighter, the admin can go onto their page and edit.	Admin clicks the Add Fighter button	Insert blank fighter into the fighter table, e.g. [John Smith, United Kingdom, Welterweight, 6'0, default picture]	Blank fighter inserted into the database	11.1
On the individual fighter editing page, when the admin clicks add fight adds a blank fight to the fighter list that you can edit with dropdown boxes and such	Admin clicks the Add fight button	Insert a blank fight where one of the fighters is the fighter whose page we are editing. When the admin enters the ids of each fighter in and then the method, put that info into fighter fights.	Insert a blank fight into the database	11.2
Insert every event since the given date into the database	Given Event number to import from Admin clicks the import button	Firstly, check that the UFC API is available; if not, then open a web page. If it is, iterate through the UFC event numbers until the UFC API can't generate a response. For	UFC Events imported onto the system or web redirect to the UFC API download page	12.1

		each of the event numbers save the location and the date.		
Insert every fight since a given date and Add it to the database	On the completion of the import of the events	Use the UFC API to chronologically iterate through every event since the given event and add each fight on the card into the fights table, including the fighters on the event, the event number, who won and how they won.	UFC fights on the fighter table and results onto fighter fights table	12.2
Link every fight to a fighter and build the fighter's profile.	Fights table with UFC import	Now go through each fight chronologically again but now gather data from the API about each fighter in each fight. Gather their name, nationality, associated gym, birthday, height and weight class.	UFC fighters on the fighter table	12.3
Run Glicko Scores	Fights and fighter tables	Do sub-problem 1-17 as if you did the editing manually	ELOs and other prob stats	12.4
Find duplicates in the data, tell the user and make them delete it before all the updates are made (56)	All available, user inputted, data (promotion/weight in belts, all the content in Events, Method/EndRound/Endtime/Title in fights, Name/Weight/Birthdate/Gym in fighters, corner, and result in fighterfights)	Use an SQL GROUP BY query to figure out fighters that have the same [name and birthday] (the chance of this happening is one in 18 million, which we are going to assume is statistically impossible) Use another GROUP BY to figure out events that are called the same thing and happen at the same time, Use another GROUP BY to figure out fights that have the same opponents and happen on the same event, Use another GROUP BY to find belts that are held by the same person in the same weight class (I'm not doing just weight class because you get interim champions that also get a belt in a division sometimes) Put these in a 2d list, their cell containing the table and their primary key	Display the list we just made to the admin when they try to submit the data they just made in order the make them correct everything. If it's a duplicate in FighterFights, refer them to the fights that they input instead of the FighterFights table, which they can't directly manipulate.	13.1
Find incorrectly formatted data, and show the user what's wrong and make them delete it	All available, user inputted, data (weight in belts, all the content in Events,	Do lookup checks for exclusively formatted fields (dates in birthdate and event date are in dd/mm/yyyy format, Method is either KO/SUB/DEC, Endtime is in a m:ss	Display the list to the admin when they try to submit changes.	13.2

before the updates are made	Method/EndRound/Endtime/Title in fights, Name/Weight/Birthdate/Gym in fighters, fighterfights result and corner) When the admin makes updates to the data and clicks the submit button	format, Corner and result is A/Red or B/Blue. Title in fights is either Y/N/RETIRED) If we find an error, put the error into a 2d list which is formatted like [[Error, Table, ID(if missing, just put null)]...]		
Find data of unnecessary length and make the admin delete it before updates are made.	All available, user inputted, data (weight in belts, all the content in Events, Method/EndRound/Endtime/Title in fights, Name/Weight/Birthdate/Gym in fighters, fighterfights result and corner) When the admin makes updates to the data and clicks the submit button (query results from previous subproblem ideally)	Do length checks where appropriate (Names up to 60 to account for cultural variations and big names, Weight classes up to 30 for long weight classes like women's super-bantamweight) If we find an error, put the error into a 2d list which is formatted like [[Error, Table, ID (if missing, just put null)] ...]	Display the list to the admin when they try to submit changes.	13.3
Do all necessary presence checks	Fighter ids for fights when entered, event Ids for fights, userID and fighterID in approvals, fighterID and fightID in fighterfights, all other primary keys and foreign keys, ELO Rating Deviation, Volatility in fighters) When the admin makes updates to the data and clicks the submit button	Do presence checks for everything we are manipulating (All FKs and PKs, Weight class wherever, Event dates, Fight methods, Not fight title in fights because not all fights are for titles, corner and result in fighterfights, names) If we find an error, put the error into a 2d list which is formatted like [[Error, Table, ID(if missing, just put null)]...]. If it's an error with the ELO rows then their Elo will be reverted to the default value	Display the list to the admin when they try to submit changes.	13.4
Make sure all data is the type it's supposed to be	All available, user inputted, data (weight in belts, all the content in Events, Method/EndRound/Endtime/Title in fights,	Do type checks (Weight classes are text only, EndRound is in one-digit format) If we find an error, put the error into a 2d list which is formatted like [[Error, Table, ID(if missing, just put null)]...]	Display the list to the admin when they try to submit changes.	13.5

	Name/Weight/Birthdate/Gym in fighters, fighterfights result and corner) When the admin makes updates to the data and clicks the submit button			
When the plus button is clicked (if not already), store the user's vote in the database	User clicks the plus button next to the fighter rating on the fighter page	Check using SQL select where fighter id=fighter on page and userid = logged in user to make sure that the user hasn't already voted the same way. If they haven't, add a record to the fighter's approval rating with a plus one	User sees the rating percentage go up	14.1
When the minus button is clicked (if not already), store the user's vote in the database	User clicks the minus button next to the fighter rating	Check using SQL select where fighter id=fighter on page and userid = logged in user to make sure that the user hasn't already voted the same way. If they haven't, add a record to the fighter's approval rating with a minus one	User sees the rating percentage go down	14.2
When the user brings up the fighter's page, view the approval rating as a percentage positive if mostly positive or a percentage negative if mostly negative	Approvals Table	SELECT all the approvals where the fighterID matches the fighter whose profile we are looking at. Make a total amount of user votes and then divide 100 by that number. Multiply that number by the number of fighters who give a positive rating. If the percentage is less than 50, then invert it (100 - the percentage) and display it as **% negative. If it's more than 50, display it as **% positive	The percentage rating is shown to the user	14.3
When the user uses the odds calculation, match each answer the user put on the survey to an attribute of the fighter	Users table Calculate odds button	Search for and calculate both fighters' submission rate, and match the number of subs to submission preference Search for and calculate both the TKO/KO rate of both fighters, and match the number of KOs/TKOs to the TKO/KO preference Search for and calculate the decision rate of both fighters and match it to the decision preference	Matched up rates in a list, ready to calculate for the entertainment test	15.1

Go through each of the attributes relative to the average fighter to figure out what they do more than average (aforementioned), and both fighters are being calculated (this part happens when the user calculates the odds).	Match up rates in a list Methods in fights table	Total the number of fights that finish by each method in fights and divide them by the amount of fights to find the average amount of each method. Give both fighters their own individual score of 100% Check fighters A and B's submission rate against the average submission rate. If they have an above-average submission rate, the formula is $new = current - 8.325(5 - x)$ With x being the number of wins the fighter has had in the method (for this example, it's submissions) Repeat this process for TKO/KOs and decisions	Display the entertainment score underneath the odds	15.2
Calculate the average score of both fighters and output that as the entertainment score	Fighter A's entertainment value Fighter B's entertainment value	Find the mean of these two values	Display the entertainment value as a label	15.3
Check for impossible fight outcomes (fight lasts 0s)	Fights table User clicks submit	SELECT all fights where the length is equal to or less than 1s, and put all of these into the same 2d list as the errors with their cell structured as [[anomaly, table, id] ...]	Output the errors to the admin when they try to submit the table	16.1
Check that every fighter has fought in amounts that are statistically possible (fewer than 12 fights a year)	FighterFights table User clicks submit	For each year, SELECT by unioning and grouping the fighters who have fought more than 12 times in a year, and put all of these into the error list, structured in [[anomaly, table, id] ...]	Output the errors so the admin can fix them when they try to submit the table	16.2
When the user navigates to another page or is redirected, specify that all other pages are hidden away from view	User navigates to another page	Whenever the user navigates, a function will be invoked with the parameter of the page they want to go to. If you are not on the page in that parameter, you are hidden. If you are, then you are shown. It will also make a Boolean variable linked to the page in a list (each page will have its own variable, I will structure it like [[page, status]...])	The user goes to the page they were meant to navigate to.	17.1
Make sure that all page-specific processes don't happen when the	User navigates to another page	Ensure that all page-specific processes that aren't supposed to happen when closed are wrapped in an if function based on 17.1 (we	Page-specific processes don't happen when they're not supposed to	17.2

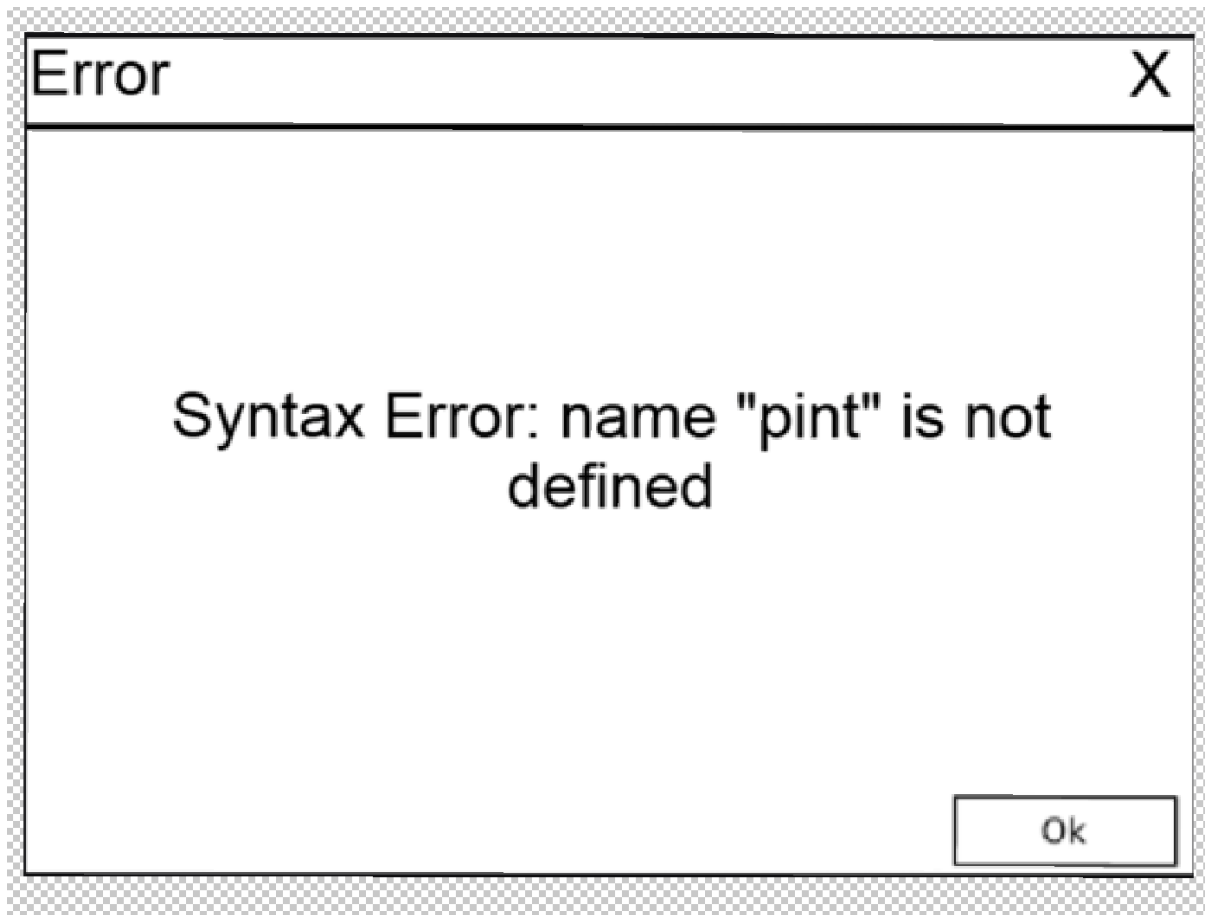
user is away unless specified otherwise		know the index already, so no need for searching)		
When the user clicks a belt button on the edit fighter list page, a list of belts automatically comes up with each holder and division shown (auto-filled from ID)	Belts table User clicks the "Edit belts..." button	Popup dialogue box. Clear the dialogue table and then iterate through with belt data. When iterating, replace fighterID with the fighter's name.	Belts pop-up that the admin can edit	18.1
The admin can add new belts	Admin clicks the plus button under the belt list Belts table	Insert blank belt into the belts table and regen table (iteration)	New belt in the table	18.2
The admin can delete belts	Admin clicks a belt and then clicks the minus button Belts table Fights table	Find the ID of the belt that the admin just clicked. Remove this belt from the database.	One less belt	18.3
If a belt is deleted, the title fight status of the belt's fights will be turned to retired	Belts table Fights table	Go through the fights table and find all fights in the division that the belt was for and set the title fight status to RETIRED	All title fights under the retired belt are marked as RETIRED	18.4
The admin can edit the holder of the belt	Admin clicks on and edits the fighter holding the belt, and then clicks off or presses ok Fighter table	If the fighter's name entered is a number, then assume that's the ID. Find the fighter ID from the fighter's name and autofill. If there are multiple fighter names, then let the user know and blank the edit.	Change put into the database or an error message tells user to "find individual fighter ID"	18.5
Open a pop-up window	User or admin gets a code error	Open a pop-up window as a dialogue box	Popup window opens	19.1
Declare the error in the pop-up window	Exception	Display the exception in the pop-up	The error is displayed	19.2
Figure out how many weight classes there are	Admin clicks submit and validate button Belts table	If a weight class doesn't have a belt, do a SELECT DISTINCT SQL function on the weight classes, which will show me all of the individual weight classes. Then, for each of these weight classes, search through the belts table. If a weight class doesn't have a belt, put the results of this query into a list.	List of weight classes	20.1
For each weight class, find the last declared title fight	List of weight classes Fights table (title,	SELECT IDs from fights where it's a title fight and it's the weight class we are looking at. Order by date. Look at the most recent title fight and get the fight ID of it. Put this into a list which looks like	List of the most recent title fights for each weight class.	20.2

		[fightID,weightclass]. Iterate through each weight class.		
Set the belt of the weight class to belong to the last winner of the weight class	Fighterfights table Recent title fights list	Look through the fighterfights table to figure out which corner won, get that fighter's ID and then add a belt onto the belts table based on their weight class and ID	Automatic new belt	20.3
If the user is an admin, when they click the View as User button redirect to the leaderboard	Admin clicks the View as user button	Redirect to the leaderboard using the page transfer processes	Admin gets to see the leaderboard and user pages	21.1
If the user is an admin, display an admin icon on the sidebar	Admin clicks the View as user button	If the user is an admin, put an admin icon on the sidebar that they can click on and go back to the edit fighter list page with	Admin has returned to the admin pages	21.2
When the admin makes an edit to the data, toggle in the system that they have.	Admin makes an edit	Toggle an editMade Boolean that will be triggered whenever an edit is made.	editMade bool is triggered	22.1
If the admin has made an edit, when they exit, submit the data and run algorithms as usual	Admin tries to exit the program	If the editMade Boolean is triggered, then go through all the checks, processing and validations you would do if the admin were to click submit	Error messages if the admin has messed up editing, auto belt assignment and ELO algorithms ran	22.2

User Interface

Don't be deceived by the title, you need to design every form you will be creating, including error messages. These need to be fully annotated, you will be showing these to your peers, they should have enough detail to be able to pick up your idea and program it. You should link these with the objectives in the design or your decomposition tasks.

General exceptions or other code errors:

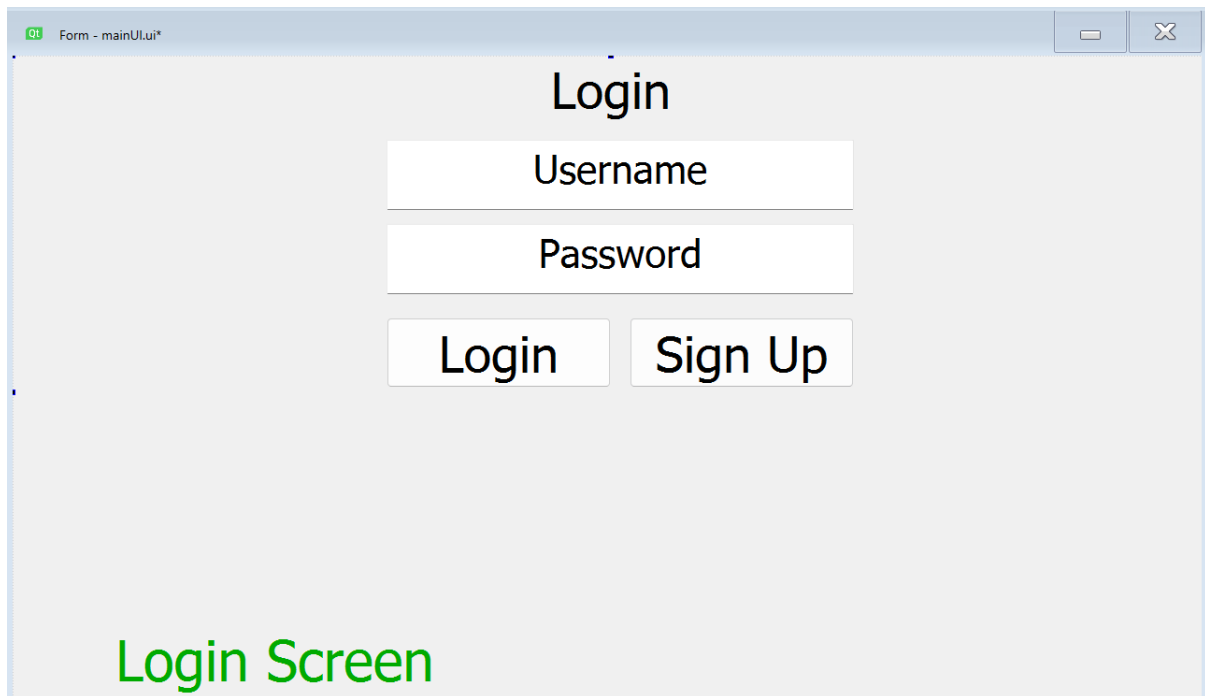


The only content this pop-up will have is the pure exception.

Login

*Annotations in green

You will be able to type in your username and password into the two text boxes and then click login if you already have an account, but if you don't, you can click sign up.

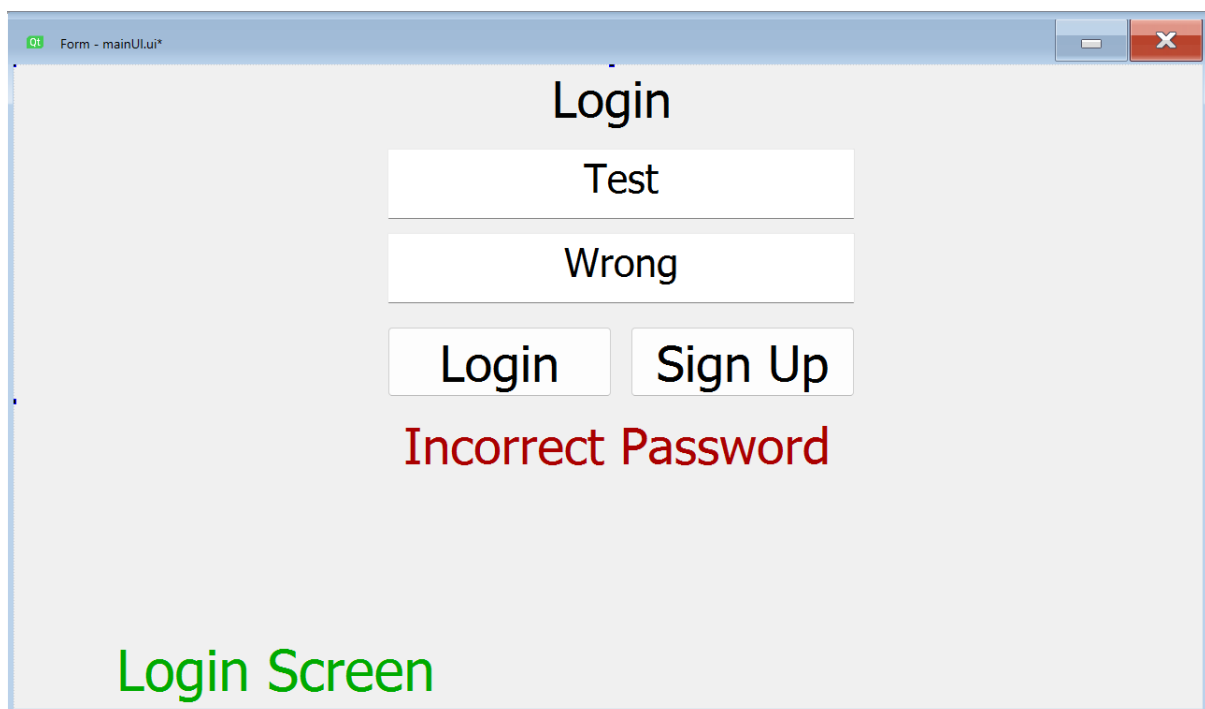


Qt Form - mainUI.ui*

Login

Login Screen

When you login with a wrong password you will get a screen like this:



Qt Form - mainUI.ui*

Login

Incorrect Password

Login Screen

If the account doesn't exist you will get this message:



Qt Form - mainUI.ui*

Login

DoesntExist

Password

Login Sign Up

Account doesn't exist

Login Screen

This is a screenshot of a Qt application window titled 'Form - mainUI.ui*'. The window has a light gray background and a blue title bar with standard Qt window controls (minimize, maximize, close). The main content area contains a login form. At the top, the word 'Login' is centered in a large black font. Below it are two white input fields with black borders; the first contains the text 'DoesntExist' and the second contains 'Password'. Underneath these fields are two white buttons with black borders, labeled 'Login' and 'Sign Up'. Below the buttons, the text 'Account doesn't exist' is displayed in a red font. At the bottom left of the window, the text 'Login Screen' is written in a green font.

If you try and sign up with a username that already exists, you will get this screen:



Qt Form - mainUI.ui*

Login

DoesExist

Password

Login Sign Up

Username already exists

Login Screen

This is a screenshot of a Qt application window titled 'Form - mainUI.ui*'. The window has a light gray background and a blue title bar with standard Qt window controls (minimize, maximize, close). The main content area contains a login form. At the top, the word 'Login' is centered in a large black font. Below it are two white input fields with black borders; the first contains the text 'DoesExist' and the second contains 'Password'. Underneath these fields are two white buttons with black borders, labeled 'Login' and 'Sign Up'. Below the buttons, the text 'Username already exists' is displayed in a red font. At the bottom left of the window, the text 'Login Screen' is written in a green font.

Form - mainUI.ui*

Login

Username box DoesExist

Password box Password

Login button Login Signup Button Sign Up

Username already exists

Error message

Login Screen screen annotation (not seen by admin/user, for development)

If you press Sign Up, you will be redirected to the survey screen:

Survey

This is what you start with:

Form - mainUI.ui*

Survey

"I like submissions"

Disagree ★ ★ ★ ★ ★ Agree

"I like knockouts"

Disagree ★ ★ ★ ★ ★ Agree

"I enjoy a long fight"

Disagree ★ ★ ★ ★ ★ Agree

Survey

Submit and Sign Up

Each star row is adjustable, and each star is clickable and autofills up.

The screenshot shows a web browser window titled "Form - mainUI.ui". The page has a light gray background. At the top center is the title "Survey" in a large, bold, black font. Below the title are three rows of survey items. Each row consists of a label "Disagree" on the left, a set of five stars in the middle, and the label "Agree" on the right. The first row is for the statement "I like submissions" and has the first two stars filled black. The second row is for "I like knockouts" and has the first three stars filled black. The third row is for "I enjoy a long fight" and has the first star filled black. At the bottom left of the form area, the word "Survey" is written in a green, sans-serif font. At the bottom center, there is a white button with the text "Submit and Sign Up" in black. The browser window has standard OS controls (minimize, maximize, close) in the top right corner.

This screenshot is an annotated version of the survey form from the previous image. It includes red annotations to explain the functionality of the UI elements. A red rectangular box highlights the five stars of the first row, with the text "Star elements determine data from 1-5" written in red to its right. Another red rectangular box highlights the "Submit and Sign Up" button at the bottom, with the text "Redirects to leaderboard" written in red to its right. The rest of the form, including the survey items and the "Survey" label, remains the same as in the previous image.

Fighters are selected from a dropdown box

Form - mainUI.ui*



Dricus Du Plessis
Israel Adesanya
Nassourdine Imavov
Micheal Page
Robert Whittaker



Michael Page

Calculate Odds

When you select a fighter from the dropdown boxes, their picture will appear

Form - mainUI.ui*

Box will display image of fighter A when selected



Dricus Du Plessis

Box will display image of fighter B when selected



Michael Page

Drop-down boxes will show all fighters if the opposite fighter hasn't been selected. If they have then they will show all the fighters from the opposite fighter's weight class.

Calculate odds button runs odds calculation/entertainment algorithms

Calculate Odds

(When clicking the Calculate odds button)

Form - mainUI.ui*


Dricus Du Plessis


Michael Page

Dricus du plessis at 75% advantage.
Probably entertaining (65%)

Calculate Odds

Result of calculations

(When clicking the button when one or both fighters are missing)

Form - mainUI.ui*




Michael Page

Please enter two fighters.

Calculate Odds

Modify the fighter list

The screenshot shows a web application interface titled 'Form - mainUI.ui'. It features a 'Search For A Fighter...' section with a table of 21 fighters. The table columns are: fighterID, name, weight class, gym, elo rating, volatility, rating deviation, and image URL. The 'Modify Fighters' panel on the right is currently empty, showing only a 'Fighter Name' label and three large empty input fields for scores (0, 0, 0). At the bottom, there are buttons for 'Add Fighter', 'Delete Fighter', and 'Submit And Validate'.

fighterID	name	weight class	gym	elo rating	volatility	rating deviation	image URL
1	Jon Jones	Heavyweight	Jackson-Wink M	2250	0.06	45	https://wikimed..
2	Islam Makhache	Lightweight	American Kickbc	2210	0.05	35	https://wikimed..
3	Alex Pereira	Light Heavyweig	Teixeira MMA	2195	0.07	32	https://wikimed..
4	Ilia Topuria	Featherweight	Climent Club	2180	0.06	38	https://wikimed..
5	Tom Aspinall	Heavyweight	Team Kaobon	2160	0.06	34	https://wikimed..
6	Merab Dvalishvili	Bantamweight	Syndicate MMA	2140	0.05	33	https://wikimed..
7	Belal Muhamma	Welterweight	Roufusport	2110	0.05	40	https://wikimed..
8	Dricus Du Plessi	Middleweight	Team CIT	2125	0.07	36	https://wikimed..
9	Alexandre Pantc	Flyweight	American Top Te	2090	0.06	35	https://wikimed..
10	Max Holloway	Featherweight	Gracie Technics	2080	0.05	30	https://wikimed..
11	Charles Oliveira	Lightweight	Chute Boxe	2065	0.08	31	https://wikimed..
12	Sean O'Malley	Bantamweight	The MMA Lab	2050	0.07	34	https://wikimed..
13	Zhang Weili	Strawweight	Black Tiger Figh	2100	0.06	42	https://wikimed..
14	Valentina Shevcl	Flyweight	Tiger Muay Thai	2095	0.05	38	https://wikimed..
15	Arman Tsarukya	Lightweight	American Top Te	2070	0.06	33	https://wikimed..
16	Khamzat Chima	Middleweight	Akhmat Fight Cli	2150	0.09	45	https://wikimed..
17	Shavkat Rakhm	Welterweight	Kill Cliff FC	2130	0.06	39	https://wikimed..
18	Robert Whittake	Middleweight	Gracie Jiu-Jitsu	2010	0.05	31	https://wikimed..
19	Magomed Ankla	Light Heavyweig	Gorets	2085	0.05	37	https://wikimed..
20	Alexander Volka	Featherweight	City Kickboxing	2040	0.06	35	https://wikimed..
21	Dustin Poirier	Lightweight	American Top Te	2005	0.06	32	https://wikimed..

Start screen will look something like this.

This annotated version of the screenshot highlights specific areas with red text and boxes. The 'Search For A Fighter...' section is labeled '(leaderboard)'. The table of fighters is labeled 'Fighters table'. The 'Modify Fighters' panel is labeled '(configuration popup)'. The 'Fighter image' placeholder is labeled 'Fighter image'. The 'Add Fighter' button is labeled 'Add Fighter'. The 'Delete Fighter' button is labeled 'Delete Fighter'. The 'Submit And Validate' button is labeled 'Submit And Validate'.

When the fighter clicks add fighter it will add a dummy fighter for the admin to edit.

Form - mainUI.ui

Search For A Fighter...

Search

fighterID	name	weight class	gym	elo rating	volatility	rating deviation	image URL
1	Jon Jones	Heavyweight	Jackson-Wink M	2250	0.06	45	https://wikimed..
2	Islam Makhache	Lightweight	American Kickbc	2210	0.05	35	https://wikimed..
3	Alex Pereira	Light Heavyweig	Teixeira MMA	2195	0.07	32	https://wikimed..
4	Ilia Topuria	Featherweight	Climent Club	2180	0.06	38	https://wikimed..
5	Tom Aspinall	Heavyweight	Team Kaobon	2160	0.06	34	https://wikimed..
6	Merab Dvalishvili	Bantamweight	Syndicate MMA	2140	0.05	33	https://wikimed..
7	Belal Muhamma	Welterweight	Roufusport	2110	0.05	40	https://wikimed..
8	Dricus Du Plessi	Middleweight	Team CIT	2125	0.07	36	https://wikimed..
9	Alexandre Panto	Flyweight	American Top Te	2090	0.06	35	https://wikimed..
10	Max Holloway	Featherweight	Gracie Technics	2080	0.05	30	https://wikimed..
11	Charles Oliveira	Lightweight	Chute Boxe	2065	0.08	31	https://wikimed..
12	Sean O'Malley	Bantamweight	The MMA Lab	2050	0.07	34	https://wikimed..
13	Zhang Weili	Strawweight	Black Tiger Figh	2100	0.06	42	https://wikimed..
14	Valentina Shevcl	Flyweight	Tiger Muay Thai	2095	0.05	38	https://wikimed..
15	Arman Tsarukya	Lightweight	American Top Te	2070	0.06	33	https://wikimed..
16	Khamzat Chima	Middleweight	Akhmat Fight Cl	2150	0.09	45	https://wikimed..
17	Shavkat Rakhm	Welterweight	Kill Cliff FC	2130	0.06	39	https://wikimed..
18	Robert Whittake	Middleweight	Gracie Jiu-Jitsu	2010	0.05	31	https://wikimed..
19	Magomed Ankai	Light Heavyweig	Gorets	2085	0.05	37	https://wikimed..
20	Alexander Volka	Featherweight	City Kickboxing	2040	0.06	35	https://wikimed..
21	John Smith	Welterweight		1500	0.05	35	

Blank fighter appears at bottom of table

View As User...Configure

Modify Fighters

Fighter Name

000

Add FighterDelete Fighter

Submit And Validate

If the user selects a fighter they will get a base overview of them.

Form - mainUI.ui

Search For A Fighter...

Search

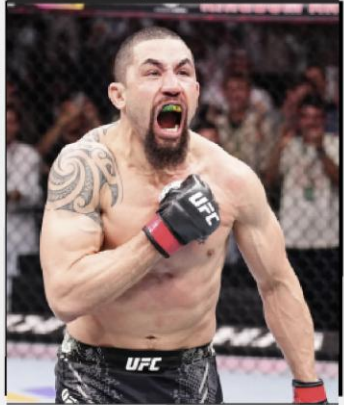
fighterID	name	weight class	gym	elo rating	volatility	rating deviation	image URL
1	Jon Jones	Heavyweight	Jackson-Wink M	2250	0.06	45	https://wikimed..
2	Islam Makhache	Lightweight	American Kickbc	2210	0.05	35	https://wikimed..
3	Alex Pereira	Light Heavyweig	Teixeira MMA	2195	0.07	32	https://wikimed..
4	Ilia Topuria	Featherweight	Climent Club	2180	0.06	38	https://wikimed..
5	Tom Aspinall	Heavyweight	Team Kaobon	2160	0.06	34	https://wikimed..
6	Merab Dvalishvili	Bantamweight	Syndicate MMA	2140	0.05	33	https://wikimed..
7	Belal Muhamma	Welterweight	Roufusport	2110	0.05	40	https://wikimed..
8	Dricus Du Plessi	Middleweight	Team CIT	2125	0.07	36	https://wikimed..
9	Alexandre Panto	Flyweight	American Top Te	2090	0.06	35	https://wikimed..
10	Max Holloway	Featherweight	Gracie Technics	2080	0.05	30	https://wikimed..
11	Charles Oliveira	Lightweight	Chute Boxe	2065	0.08	31	https://wikimed..
12	Sean O'Malley	Bantamweight	The MMA Lab	2050	0.07	34	https://wikimed..
13	Zhang Weili	Strawweight	Black Tiger Figh	2100	0.06	42	https://wikimed..
14	Valentina Shevcl	Flyweight	Tiger Muay Thai	2095	0.05	38	https://wikimed..
15	Arman Tsarukya	Lightweight	American Top Te	2070	0.06	33	https://wikimed..
16	Khamzat Chima	Middleweight	Akhmat Fight Cl	2150	0.09	45	https://wikimed..
17	Shavkat Rakhm	Welterweight	Kill Cliff FC	2130	0.06	39	https://wikimed..
18	Robert Whittake	Middleweight	Gracie Jiu-Jitsu	2010	0.05	31	https://wikimed..
19	Magomed Ankai	Light Heavyweig	Gorets	2085	0.05	37	https://wikimed..
20	Alexander Volka	Featherweight	City Kickboxing	2040	0.06	35	https://wikimed..
21	John Smith	Welterweight		1500	0.05	35	

fighter selected

Overview

View As User...Configure

Modify Fighters



Robert Whittaker

1915

Add FighterDelete Fighter

Submit And Validate

If the user selects delete fighter with a selected fighter, the fighter will be removed from the table.

fighterID	name	weight class	gym	elo rating	volatility	rating deviation	image URL
1	Jon Jones	Heavyweight	Jackson-Wink M	2250	0.06	45	https://wikimed..
2	Islam Makhache	Lightweight	American Kickbc	2210	0.05	35	https://wikimed..
3	Alex Pereira	Light Heavyweig	Teixeira MMA	2195	0.07	32	https://wikimed..
4	Ilia Topuria	Featherweight	Cilment Club	2180	0.06	38	https://wikimed..
5	Tom Aspinall	Heavyweight	Team Kaobon	2160	0.06	34	https://wikimed..
6	Merab Dvalishvil	Bantamweight	Syndicate MMA	2140	0.05	33	https://wikimed..
7	Belal Muhamma	Welterweight	Roufusport	2110	0.05	40	https://wikimed..
8	Dricus Du Plessi	Middleweight	Team CIT	2125	0.07	36	https://wikimed..
9	Alexandre Pantc	Flyweight	American Top Te	2090	0.06	35	https://wikimed..
10	Max Holloway	Featherweight	Gracie Technics	2080	0.05	30	https://wikimed..
11	Charles Oliveira	Lightweight	Chute Boxe	2065	0.08	31	https://wikimed..
12	Sean O'Malley	Bantamweight	The MMA Lab	2050	0.07	34	https://wikimed..
13	Zhang Weli	Strawweight	Black Tiger Figh	2100	0.06	42	https://wikimed..
14	Valentina Shevcl	Flyweight	Tiger Muay Thai	2095	0.05	38	https://wikimed..
15	Arman Tsarukya	Lightweight	American Top Te	2070	0.06	33	https://wikimed..
16	Khamzat Chima	Middleweight	Akhmat Fight Cli	2150	0.09	45	https://wikimed..
17	Shavkat Rakhm	Welterweight	Kill Cliff FC	2130	0.06	39	https://wikimed..
19	Magomed Ankai	Light Heavyweig	Gorets	2085	0.05	37	https://wikimed..
20	Alexander Volka	Featherweight	City Kickboxing	2040	0.06	35	https://wikimed..
21	John Smith	Welterweight		1500	0.05	35	

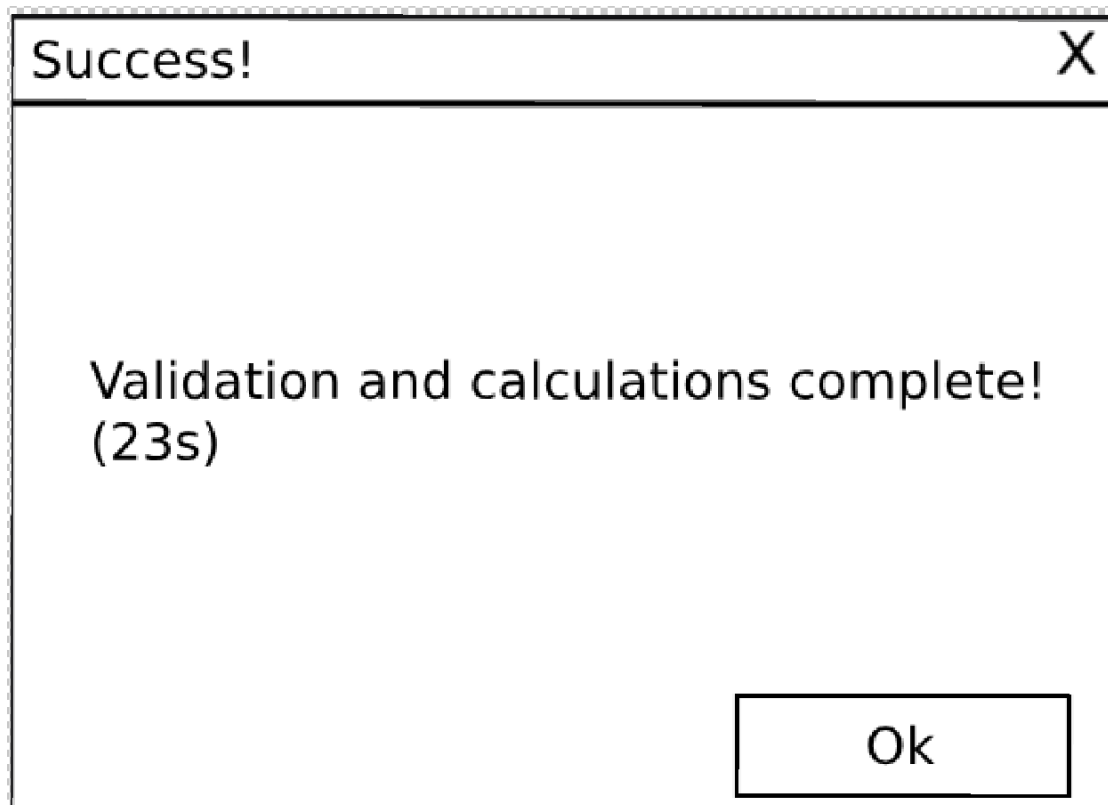
If the user clicks delete fighter and there is no fighter selected they will get this popup:

Error

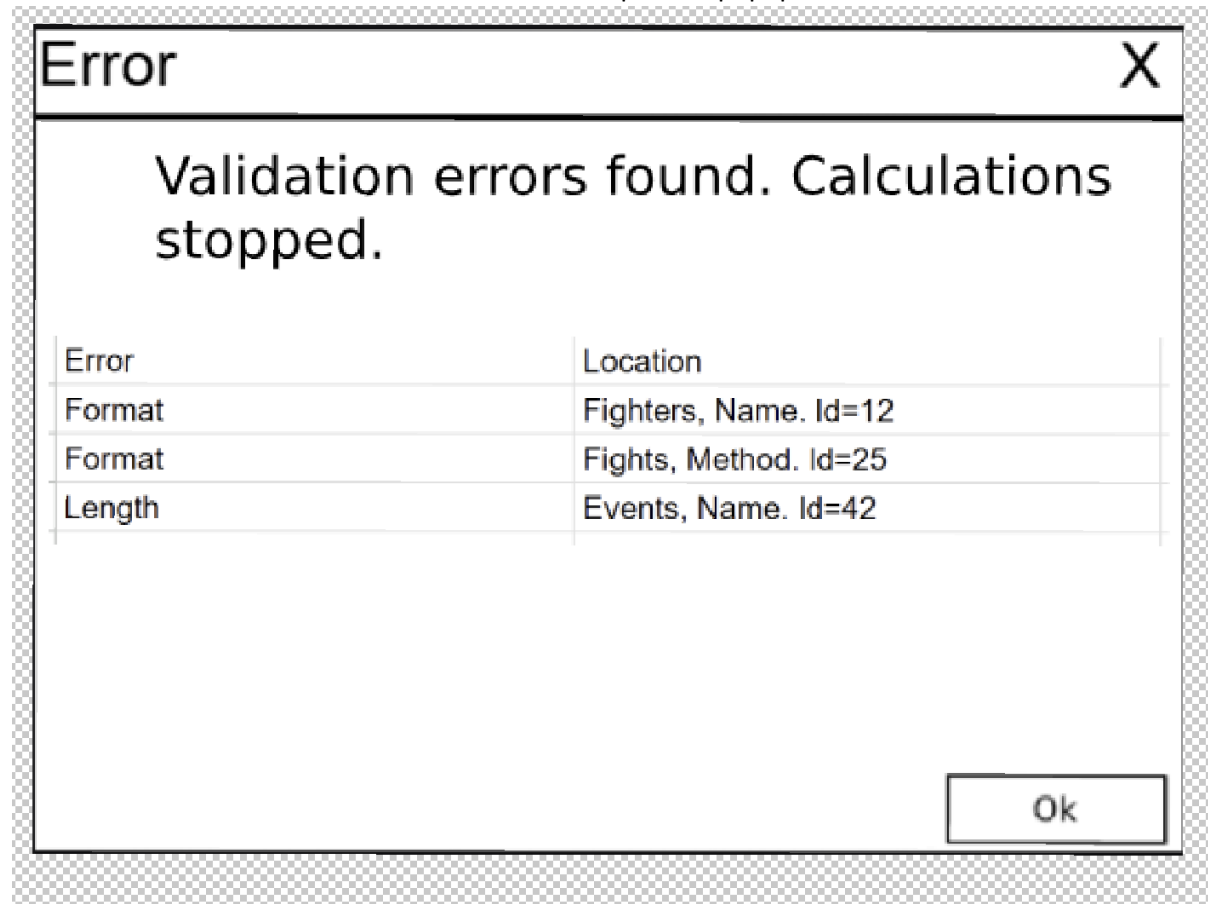
No fighter selected to delete.

Ok

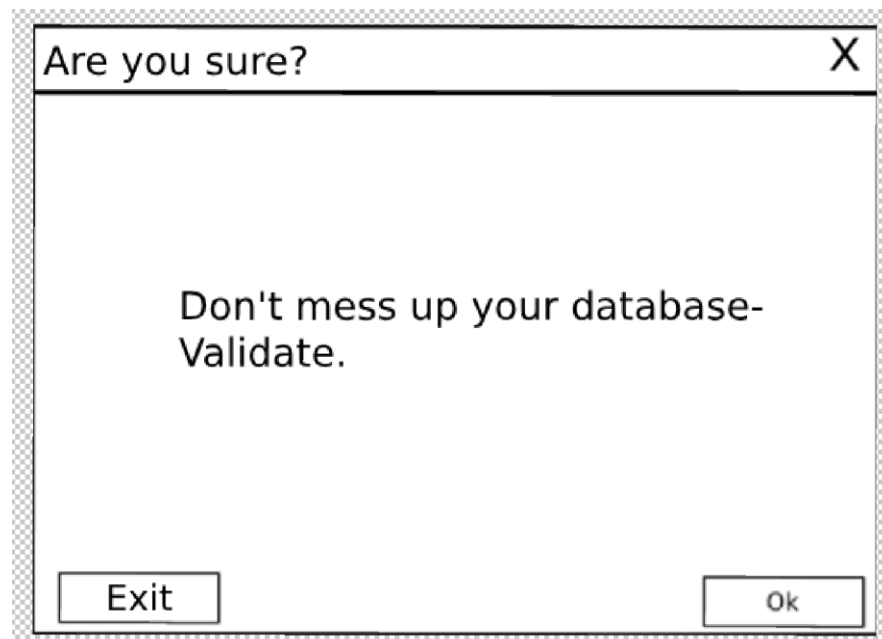
If the user clicks “submit and validate”, and everything is all good, the table will give them this message.



If there is admin-caused validation errors it will output this popup:

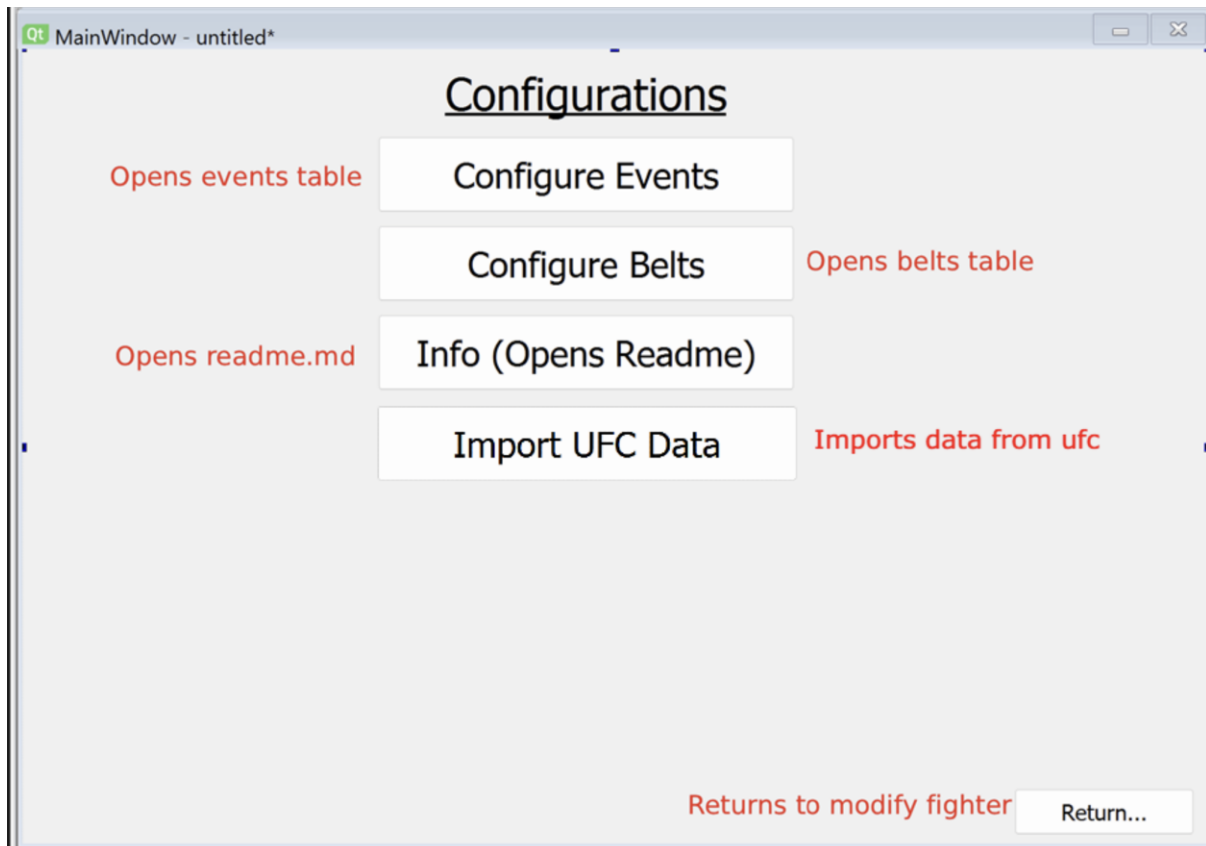


If the admin directly tries to exit the program and they've made an edit to the database, it will show them this.



Configurations (not really a page, but I want to separate it)

You can open this pop-up page from the modify fighter list page.

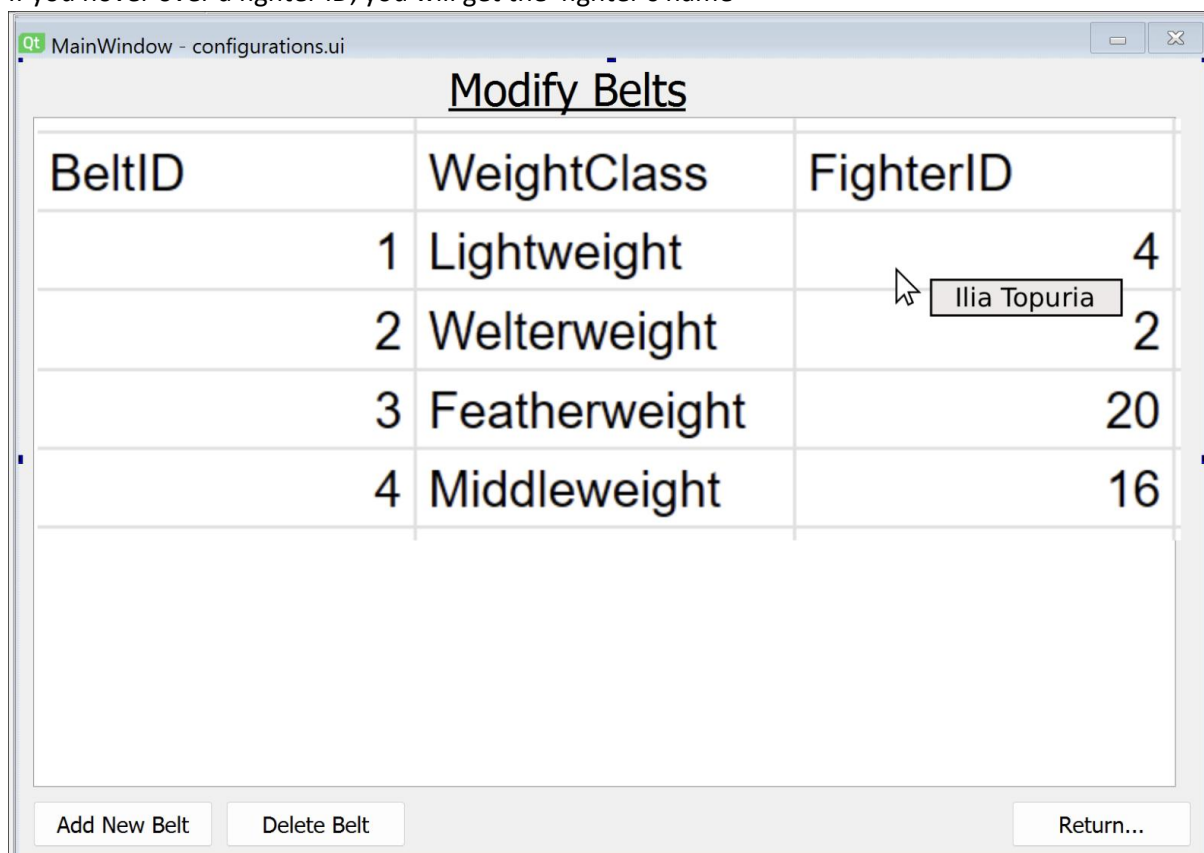


Configurations table page

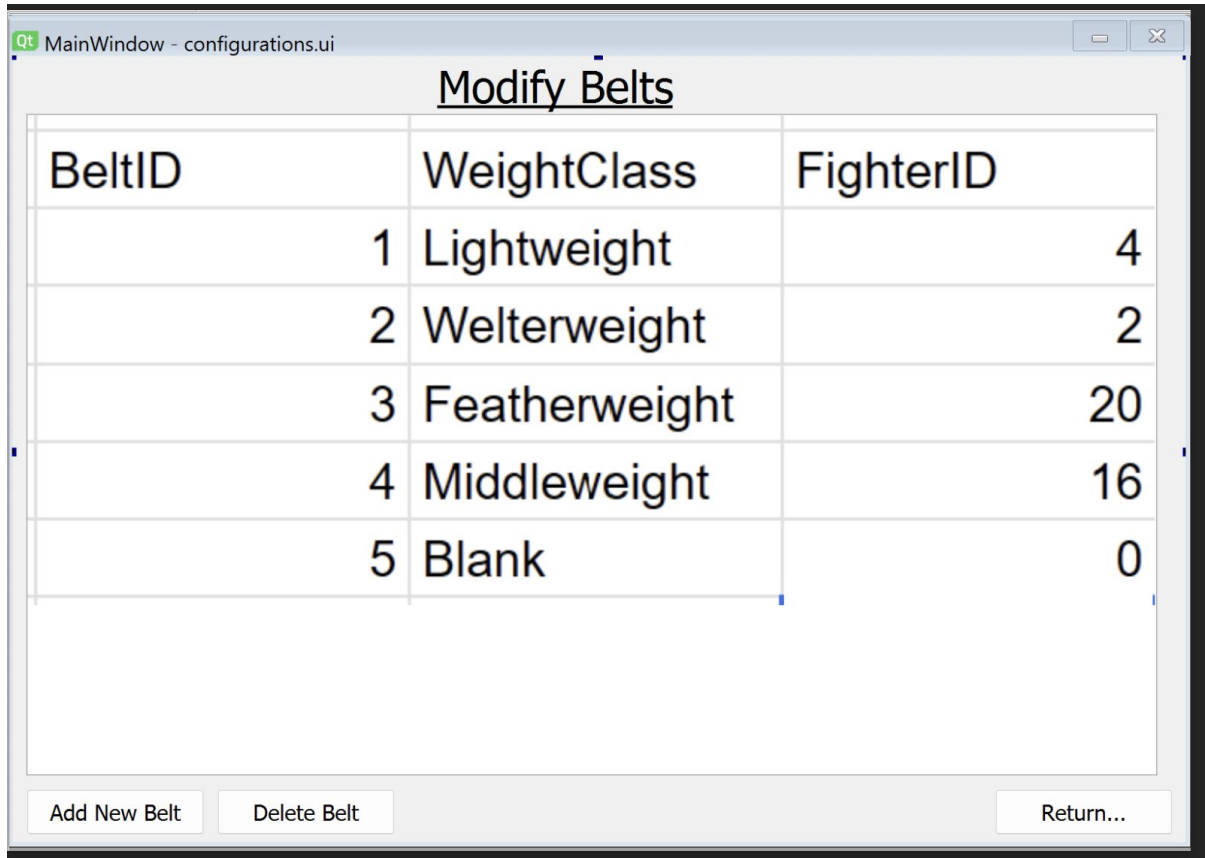
Show the data of the belts table



If you hover over a fighter ID, you will get the fighter's name



When the admin clicks add belt the ui will add a blank belt



The screenshot shows a Qt application window titled "MainWindow - configurations.ui". The window contains a form titled "Modify Belts". Inside the form is a table with three columns: "BeltID", "WeightClass", and "FighterID". The table contains five rows of data. Below the table are three buttons: "Add New Belt", "Delete Belt", and "Return...".

BeltID	WeightClass	FighterID
1	Lightweight	4
2	Welterweight	2
3	Featherweight	20
4	Middleweight	16
5	Blank	0

If you click delete belt it will delete whatever belt you had selected.

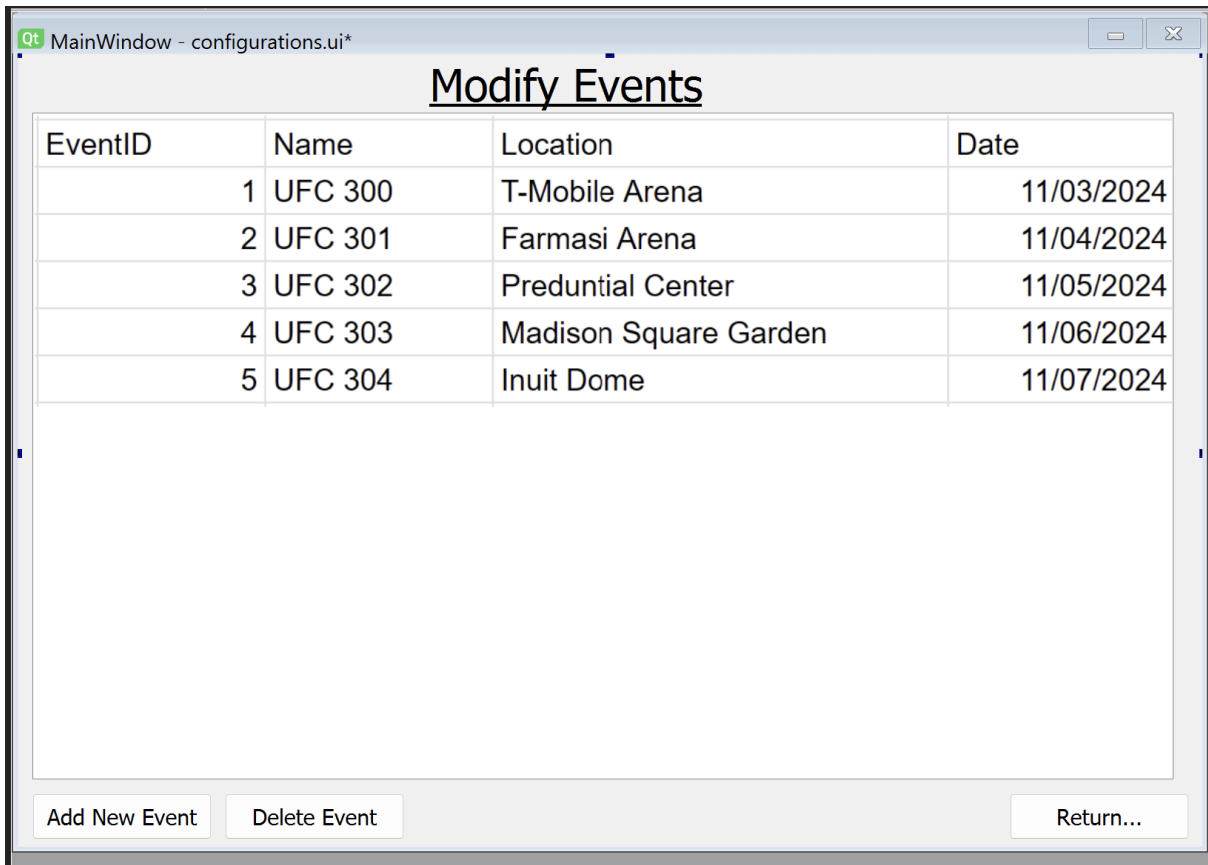


The screenshot shows the same Qt application window, but the "Delete Belt" button has been clicked. The table now only contains four rows of data, as the row with BeltID 5 has been removed. The buttons "Add New Belt", "Delete Belt", and "Return..." are still present at the bottom.

BeltID	WeightClass	FighterID
1	Lightweight	4
2	Welterweight	2
3	Featherweight	20
4	Middleweight	16

(IMPORTANT TO MENTION THAT BOTH TABLES ARE EDITABLE EXCEPT FOR THE IDs)

Modify events is very similar



The screenshot shows a Qt application window titled "MainWindow - configurations.ui*" with a standard window control bar. The main content area is titled "Modify Events" in a large, bold, black font. Below the title is a table with four columns: "EventID", "Name", "Location", and "Date". The table contains five rows of data. Below the table is a large, empty rectangular area. At the bottom of the window, there are three buttons: "Add New Event", "Delete Event", and "Return...".

EventID	Name	Location	Date
1	UFC 300	T-Mobile Arena	11/03/2024
2	UFC 301	Farmasi Arena	11/04/2024
3	UFC 302	Preduntial Center	11/05/2024
4	UFC 303	Madison Square Garden	11/06/2024
5	UFC 304	Inuit Dome	11/07/2024

Buttons: Add New Event, Delete Event, Return...

Add blank event

MainWindow - configurations.ui*

Modify Events

EventID	Name	Location	Date
1	UFC 300	T-Mobile Arena	11/03/2024
2	UFC 301	Farmasi Arena	11/04/2024
3	UFC 302	Preduntial Center	11/05/2024
4	UFC 303	Madison Square Garden	11/06/2024
5	UFC 304	Inuit Dome	11/07/2024
6	BLANK		BLANK

Add New Event Delete Event Return...

Leaderboard

This is what a baseline user will see if they open the leaderboard.

Form - mainUI.ui*

Odds Checker

Search name... (Levenshtein) Search

Name	Weight class	Gym
Jon Jones	Heavyweight	Jackson-Wink MMA
Islam Makhachev	Lightweight	American Kickboxing Academy
Alex Pereira	Light Heavyweight	Teixeira MMA
Ilia Topuria	Featherweight	Climent Club
Tom Aspinall	Heavyweight	Team Kaobon
Merab Dvalishvili	Bantamweight	Syndicate MMA
Belal Muhammad	Welterweight	Roufusport
Dricus Du Plessis	Middleweight	Team CIT
Alexandre Pantoja	Flyweight	American Top Team
Max Holloway	Featherweight	Gracie Technics
Charles Oliveira	Lightweight	Chute Boxe

Scrollable table from "fighters"

This is what they will see when they select a fighter.

Form - mainULui*

odds checker button

Odds Checker

Khamzat Chimaev

15-0-0^{W/D/L}





Search name... (Levenshtein)

Search

Name	Weight class	Gym
Jon Jones	Heavyweight	Jackson-Wink MMA
Islam Makhachev	Lightweight	American Kickboxing Academy
Alex Pereira	Light Heavyweight	Teixeira MMA
Ilia Topuria	Featherweight	Climent Club
Tom Aspinall	Heavyweight	Team Kaobon
Merab Dvalishvili	Bantamweight	Syndicate MMA
Belal Muhammad	Welterweight	Roufusport
Dricus Du Plessis	Middleweight	Team CIT
Alexandre Pantoja	Flyweight	American Top Team
Max Holloway	Featherweight	Gracie Technics
Charles Oliveira	Lightweight	Chute Boxe

An admin will also have the modify fighters page available to go to.

Form - mainULui*

Manage List...

Odds Checker

Khamzat Chimaev

15-0-0^{W/D/L}



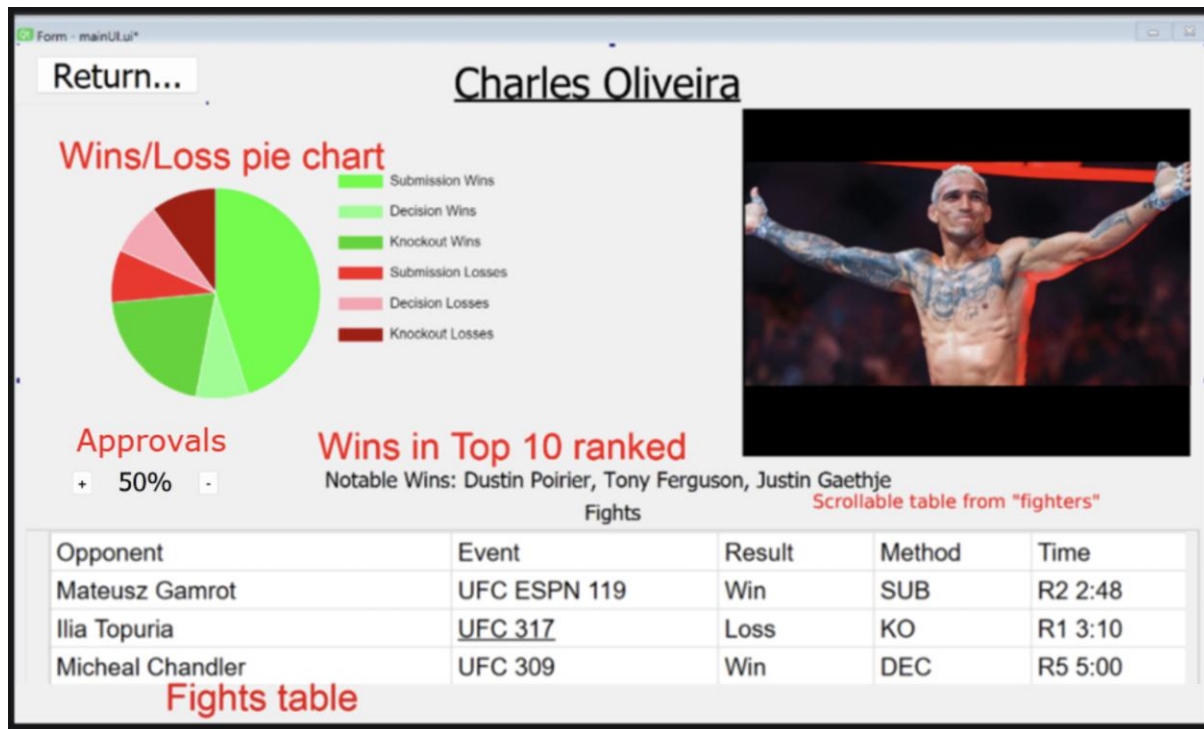


Search name... (Levenshtein)

Search

Name	Weight class	Gym
Jon Jones	Heavyweight	Jackson-Wink MMA
Islam Makhachev	Lightweight	American Kickboxing Academy
Alex Pereira	Light Heavyweight	Teixeira MMA
Ilia Topuria	Featherweight	Climent Club
Tom Aspinall	Heavyweight	Team Kaobon
Merab Dvalishvili	Bantamweight	Syndicate MMA
Belal Muhammad	Welterweight	Roufusport
Dricus Du Plessis	Middleweight	Team CIT
Alexandre Pantoja	Flyweight	American Top Team
Max Holloway	Featherweight	Gracie Technics
Charles Oliveira	Lightweight	Chute Boxe

When the user double-clicks, they will be taken to that fighter's page.



When the admin double-clicks a fighter on the modify fighter list page, they will get an alternative version of this profile, which features editing capabilities. Here is what that version looks like with no data

Form - mainULui*

Editing Fighter Name

Name

Gym

Weight Class

Birthdate

Belts: None

Image URL



Here is what it should look like in action.

Form - mainUI.ui*

Editing Charles Oliveira

Charles Oliveira

Chute Boxe

Lightweight

Birthdate 01/01/2000

Belts: None

<https://static0.thesportsterimages.com/wordpress/wp> Preview



Opponent ID	Event ID	Result	Method	Round	Time
87	98	Win	SUB	2	2:48
54	84	Loss	KO	1	3:10
32	78	Win	DEC	5	5:00
63	73	Loss	DEC	5	5:00
42	67	Win	KO	1	2:16
23	56	Loss	SUB	2	1:23

Add fightDelete Fight

Submit/Return

Data structure

For my project. I am using a relational SQL database. I have chosen to use an SQL database to store all my data. This is by far the most suitable way to store all my data for a multitude of reasons.

The reason I am using a relational database is because a relational database is well structured, making queries simpler. Also, a relational database allows all the connections between all my data to be represented efficiently with primary and foreign keys. A relational database is also good for data integrity. This is because you don't have to repetitively refer to things, like you may do with other databases, which makes the database more vulnerable to data inconsistency. Relational databases are also more secure, as the database doesn't have to return as much in a query that you could get in other databases.

The reason I am using SQL is that SQL is one of the best options for doing advanced database queries efficiently. I am going to be doing a lot of advanced calculations and queries, which SQL handles better than something like a Python flat file, for example. SQL also supports indexing. Without indexing, some of the queries I want to make will be extremely slow, especially when the database grows.

We are dealing with very valuable user data, so good security is an absolute must. SQL also helps with security since it supports simultaneous operations without messing up all the data. It also works well with hashing algorithms and requires authentication to access the database.

For management of queries, I am going to use Microsoft SQL Server. This is because Microsoft SQL server lets you deal with datatypes like dates which is really useful for when we handle event date ranges and things like that for validation's sake

Methods of Access

Entity/Field in entity	Method Of Access	SQL Query	Sub Problem	Justification
Fighter Fights	Indirect	SELECT	1.1	The algorithm indirectly accesses each fight by accessing the results in the table of all the fights
Variance, ELO, RD from Fighters	Indirect	SELECT	1.1	The algorithm indirectly accesses each fighter by accessing the Elo stats of all the fighters in the table (using the foreign keys of Fighter Fights)
Fighter ELO from Fighters	Direct	UPDATE	1.13	The algorithm directly updates the fighter they're checking's ELO (using the foreign keys from Fighter Fights)
Fighter volatility from Fighters	Direct	UPDATE	1.10	The algorithm directly updates the fighter they're checking's volatility (using the foreign keys from Fighter Fights)
Fighter rating deviation from Fighters	Direct	UPDATE	1.13	The algorithm directly updates the fighter they're checking's rating deviation using the foreign keys from Fighter Fights
Fighter stats from fighters	Indirect	SELECT	1.16	The algorithm indirectly accesses each fighter by putting them all onto a model for the admin/normal user to see
Find all fighter names from the fighters	Indirect	SELECT	4.1	Indirectly accesses each fighter by getting all their names up
Find the profile fighter's image URL from the fighters	Direct	SELECT	6.1	Directly accesses the fighter's image link using their primary key
Find fights where the profile fighter has fought in fighterfights and fights	Direct	SELECT	6.2	Directly accesses the fights that the fighter has fought in via a primary key, and then accesses the stats of those fights using a foreign key
Find Belts the profile fighter holds in belts	Direct	SELECT	6.6	Directly accesses the belts that the fighter holds via the fighter's primary key.
Find the fighters who fight in the same division as the fighter profile in fighters	Indirect	SELECT	8.1	Indirectly accesses the fighters in the same weight division as the fighter we're looking at

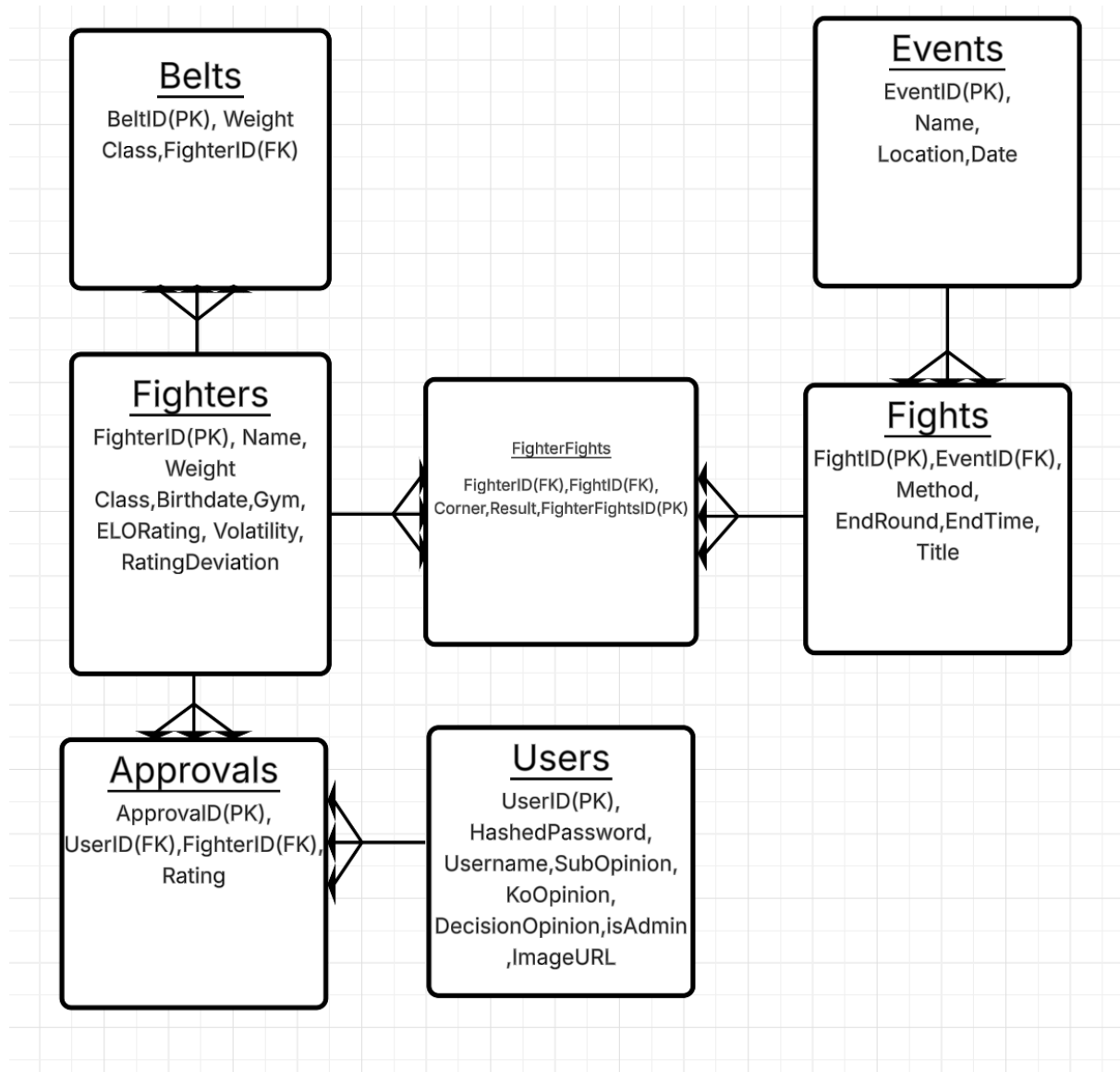
Find the fights that the top 10 fighters of the division have fought against the profile fighter in from fighters	Indirect	SELECT	8.2	Indirectly accesses the fights with fighters who are in the top 10 going against the fighter whose profile we are looking at
Find all fighters to display in the dropdown box alphabetically from fighters	Indirect	SELECT	9.1	Indirectly accesses all the names of the fighters in the leaderboard and puts them into a dropdown box
Find all fighters who fight in the weight class of the fighter selected in the other box from fighters	Indirect	SELECT	9.1	Indirectly accesses all the names of the fighters in the same weight class as the fighter in the other box
Checks the system for a duplicate username in users	Direct	SELECT	10.1	Directly accesses the one person who has the same username of the person trying to log in, in the users table, if they are there
Adds new user into users table	Direct	INSERT	10.2	Directly inserts new user with their data into users table
Logs user into their account by checking users table	Direct	SELECT	10.3	Directly accesses the hashed password of the profile the user is trying to log into in the users table using the username as the account's unique identifier
Checks if user is an admin in users table	Direct	SELECT	10.4	Directly accesses the isAdmin bool of the user's account in the users table
Updates user table with survey data	Direct	UPDATE	10.5	Directly accesses the user profile using their ID and adds their survey data to the users table
Add blank fighter onto the fighters table	Direct	INSERT	11.1	Directly inserts the blank fighter into the fighters table
Add blank fight onto the fights table	Direct	INSERT	11.2	Directly inserts the blank fight into the fights table

Autofills fights onto fighter fights	Direct	INSERT	11.2	Automatically enters info into fighter fights based off of fighter ids and outcomes
Insert a UFC event into the events table	Direct	INSERT	12.1	Directly inserts UFC events into the events table from a given date
Insert a UFC fight into the fights table	Direct	INSERT	12.2	Directly inserts UFC fights referenced to the events they are from via a foreign key
Insert the UFC fighters onto the database	Direct	INSERT	12.3	Directly inserts each fighter referenced by fights onto the database
Insert the UFC fight results onto fighter fights table	Direct	INSERT	12.3	Directly inserts each fight result referenced to a fight and two fighters
Select fighters in groups based off matching names and birthdays	Indirect	SELECT (GROUP BY)	13.1	Indirectly accesses each fighter to group them up if duplicated
Select events in groups based off of matching names and dates	Indirect	SELECT (GROUP BY)	13.1	Indirectly accesses each event to group them up if duplicated
Select fightIDS in groups from fighter fights based on matching fighters and events	Indirect	SELECT (GROUP BY)	13.1	Indirectly accesses each fighter fight record to group them up if they are a duplicate
Select weight from belts, all of events, method/end round/end time/title in fights, name/weight/birthday /Gym in fighters, result/corner in fighterFights	Indirect	SELECT (GROUP BY)	13.1	Indirectly accesses each of these records to validate them (format and range, and presence checks)

Find approval votes that are the same user and fighter	Direct	SELECT	14.1/14.2	Directly accesses a single rating by the userID and fighterID to see if the user has already voted the same way to stop them from voting twice
Add approval rating to the fighters rating (approvals)	Direct	INSERT	14.1/14.2	Directly inserts into the approvals table to add negative/positive rating to fighter rating table
Select all approval ratings to a fighter to figure out their rating (approvals)	Direct	SELECT	14.3	Directly accesses a single fighter's rating from their fighterID
Select all fight results to total up the fights a fighter has fought in (fighterfights)	Direct	SELECT	15.1	Directly accesses a fighter's results by their id
Select all fight results where a fighter has won via submission (fighterfights and fights)	Direct	SELECT	15.1	Directly accesses fights referred to by fighter id where the method is a submission
Select all fight results where a fighter has won via ko (fighterfights and fights)	Direct	SELECT	15.1	Directly accesses fights referred to by fighter id where the method is a ko
Select all fight results where a fighter has won via decision (fighterfights and fights)	Direct	SELECT	15.1	Directly accesses fights referred to by fighter id where the method is a decision
Get the opinion survey results of the user	Direct	SELECT	15.2	Directly accesses the opinion survey of the user account with the same id as the user who's using the program's id (i.e. the user)

currently using the table (Users)				
Look for fights where they last equal to or less than 1s	Indirect	SELECT	16.1	Indirectly accesses all fights that are impossibly short
Look for fighters with a fight count of more than 12	Indirect	SELECT with a SELECT GROUP BY subquery	16.2	Indirectly accesses all fights to count them up for each fighter , for every year
Bring up all belts from belts table	Indirect	SELECT	18.1	Indirectly accesses all belts to put them onto a table model
Add blank belt into belts table	Direct	INSERT	18.2	Directly insert a blank belt into the belts table
Delete belt from belts table	Direct	DELETE	18.3	Belt is directly deleted by its ID
Set title fight status of fights involving deleted belt to RETIRED (fights)	Indirect	UPDATE	18.4	Each fight is indirectly accessed to update the fights that include the title that just been retired to have the title as RETIRED
Find out how many weight classes there are in fighters	Direct	SELECT DISTINCT	20.1	Directly selects one example for each weight class
Find most recent title fight in individual weight class	Direct	SELECT	20.2	Selects one distinct title fight for each weight class (being the most recent ones)
Insert new belt with recent title fight winner onto database	Direct	INSERT	20.3	Directly inserts auto-generated belts onto table

Database design – ERD



Justification of ERD

These are the main entities that are interacted with in the system. This entity relationship diagram shows how each of them interacts with the other. The admin enters an event into the system. Every event has many fights. Many fighters have many fights, so I have an intermediate Fighter Fights table. A belt can only be held by one fighter, and a fighter can have multiple belts

The way the User interacts with the fighter's data is through their approval ratings. A user will vote a fighter up or down. One user has many Approval ratings. Many approval ratings are linked to one fighter.

Data Dictionary.

EVERY table you have identified in your ERD MUST have its own Data Dictionary!! Each one should have its own separate table.

	Table Name: Belts				
Field Name	Data Type	Description	Example	Validation	Key
BeltID	Int	Identifier for belt	12	Presence	Primary
Weight	Varchar	Weight class or just the belt title if it's a special title	Welterweight	Length	
FighterID	Int	The fighter who holds the belt	75	Lookup	Foreign

	Table Name: Events				
Field Name	Data Type	Description	Example	Validation	Key
EventID	Int	Event Identifier	3	Presence	Primary
Name	Varchar	Event name	UFC 300	Length	
Location	Varchar	Event Location	Madison Square Garden	Length	
Date	Varchar (Date)	Date of event	11/03/2024	Format/Range (dd/mm/yyyy)	

	Table Name: Users				
Field Name	Data Type	Description	Example	Validation	Key
UserID	Int	Identifier for user	32	Presence	Primary
HashedPassword	Varchar	User's Password hashed and salted	b834aac0f6...	Presence	
Username	Varchar	User's Username	24jubbt476	Presence	
SubOpinion	Int	Submission survey answer 1-5	3	Range	
KoOpinion	Int	Ko survey answer 1-5	5	Range	
DecisionOpinion	Int	Decision Survey Answer	1	Range	
isAdmin	Boolean	Checks if the user is an admin or not	False	Presence	

	Table Name: Approvals				
Field Name	Data Type	Description	Example	Validation	Key
ApprovalID	Int	Identifier for approval vote	12	Presence	Primary

UserID	Int	Identifier for user	53	Lookup	Foreign
FighterID	Int	Identifier for a fighter	24	Lookup	Foreign
Rating	Boolean	Nature of approval (+1/positive if true, -1/negative if false)	True	Type	

Table Name: Fights					
Field Name	Data Type	Description	Example	Validation	Key
FightID	Int	Identifier for a fight	20	Presence	Primary
EventID	Int	Event the fight is in	42	Lookup	Foreign
Method	Varchar	The way the fight ended	KO	Lookup	
EndRound	Int	Round the fight ended	4	Length	
EndTime	Varchar (Time)	The time the fight ended	2:32	Format/Range	
Title	Varchar	If it's a title fight, the title that the fight was fought for	Welterweight	Length	

Table Name: FighterFights					
Field Name	Data Type	Description	Example	Validation	Key
FighterFightsID	Int	Identifier for intermediate	23	Presence	Primary
FighterID	Int	Identifier for Fighter	45	Lookup	Foreign
FightID	Int	Identifier for the fight	21	Lookup	Foreign
Corner	Varchar	Corner the fighter was in	Red	Lookup	
Result	Varchar	Result of fighter in fight	W	Lookup	

Table Name: Fighters					
Field Name	Data Type	Description	Example	Validation	Key
FighterID	Int	Identifier for a fighter	24	Presence	Primary

Name	Varchar	Name of fighter	Khalil Rountree	Length	
WeightClass	Varchar	Weight class of the fighter	Light Heavyweight	Lookup	
Birthdate	Varchar	Fighter's birthday	11/04/1993	Format	
Gym	Varchar	Gym that fighter fights at	American Kickboxing Academy	Length	
ELORating	Float	ELO rating of a fighter	1500	Presence and range	
Volatility	Float	Variation in the performance of the fighter	1.2	Presence and range	
Rating Deviation	Float	Certainty of fighter's rating	50	Presence and range	
ImageURL	Varchar	URL of image of fighter		Length	

Validation

Data Validation techniques I use:

Presence check

A presence check is where you ensure that a required field has not been left empty or blank. For example, all primary keys cannot be left empty to prevent nulls.

Length check

Confirms the string data is inside of a character limit. For example, an event name can only be up to 80 characters long

Range check

Confirms that a number falls inside of a boundary. For example an opinion rating has to be from 1 to 5.

Format check

Confirms that data adheres to a specific structure. For example, the end time column in the fights table has to be in the format mm:ss or m:ss

Lookup check

Ensures that a foreign key actually exists as a primary key in the other table or if it matches a list of pre-defined values (domain lookup). For example, a gym name can only be up to 80 characters long

Type check

Confirms that an input is a specific data type. For example, an approval rating needs to be boolean

It's important to note that in actual deployment (and most definitely with the final copy of my project) In the readme it will go through all the types of validation rules I use for the admin to understand what each mean.

Data validation cases

(When I refer to a list in the error messages the contents of that list will be output as a validation error table to the admin)

Table	Field	Valid. Type	Validation Rule	Error Message
Belts	BeltID	Presence	BeltID cannot be left empty.	User:Validation Error, please contact admin. Admin:[Presence,Belts,BeltID, Name=Welterweight]
Belts	Weight	Length	Up to 60 characters long	User:Validation Error, please contact admin. Admin:[Format,Belts,Weight, BeltID={BeltID}]
Belts	FighterID	Lookup	FighterID must really exist as a fighter.	User:Validation Error, please contact admin. Admin:[Lookup,Belts,FighterID,BeltID={Fighter}]
Events	EventID	Presence	EventID cannot be left empty	User:Validation Error, please contact admin. Admin:[Presence,Events,EventID, Name={Events}]
Events	Name	Length	The name of the event cannot be longer than 80 characters	User:Validation Error, please contact admin. Admin:[Length,Events,Name, EventID={EventID}]
Events	Location	Length	The location name cannot be longer than 100 characters	User:Validation Error, please contact admin. Admin:[Length,Events,Location,

				EventID={EventID}}
Events	Date	Format/Range	The format of the date must follow dd/mm/yyyy format and can't be in the future	User:Validation Error, please contact admin. Admin:[Format,Events,Date, EventID={EventID}]
Users	UserID	Presence	UserID cannot be left empty	User:Validation Error, please contact admin. Admin:[Presence,Users,UserID, Username={Username}]
Users	HashedPassword	Presence	The password can't be left empty	User:Validation Error, please contact admin. Admin:[Presence,Users,UserID,HashedPassword, Username={Username}]
Users	Username	Presence	Username can't be left empty	User:Validation Error, please contact admin. Admin:[Presence,Users,UserID,Username, UserID={UserID}]
Users	SubOpinion	Range	The opinion ratings should be from 1-5	User:Validation Error, please contact admin. Admin:[Range,Users,UserID,SubOpinion UserID={UserID}]
Users	KoOpinion	Range		User:Validation Error, please contact admin. Admin:[Range,Users,KoOpinion, UserID={UserID}]
Users	DecisionOpinion	Range		User:Validation Error, please contact admin. Admin:[Range,Users,DecisionOpinion, UserID={UserID}]
Users	isAdmin	Presence	The admin identifier cannot be left empty	Automatically reverted to False but errors will be handled to be safe User:Validation Error, please contact admin. Admin:[Presence,Users,isAdmin, UserID={UserID}]
Approvals	ApprovalID	Presence	ApprovalID can't be left empty	User:Validation Error, please contact admin. Admin:[Presence,Approvals,ApprovalID, UserID={UserID}]
Approvals	UserID	Lookup	The user the approval is referring to must be real	User:Validation Error, please contact admin. Admin:[Lookup,Approvals,UserID, ApprovalID={ApprovalID}]
Approvals	FighterID	Lookup	The fighter the approval is referring to must be real	User:Validation Error, please contact admin. Admin:[Lookup,Approvals,FighterID, ApprovalID={ApprovalID}]
Approvals	Rating	Type	The Rating itself needs to be Boolean	User:Validation Error, please contact admin. Admin:[Type,Approvals,Rating, ApprovalID={ApprovalID}]
Fights	FightID	Presence	The fight ID cannot be left empty	User:Validation Error, please contact admin. Admin:[Presence,Fights,FightID, EventID={EventID}]
Fights	EventID	Lookup	The event the fight is on must exist	User:Validation Error, please contact admin. Admin:[Lookup,Fights,EventID, FightID={FightID}]
Fights	Method	Lookup	(normalised into uppercase) The result must be either [SUB,KO,DEC,NC,DRAW]	User:Validation Error, please contact admin. Admin:[Lookup,Fights,Method, FightID={FightID}]
Fights	EndRound	Length	The number must be up to 5 rounds	User:Validation Error, please contact admin. Admin:[Length,Fights,EndRound, FightID={FightID}]
Fights	EndTime	Format/Range	The time must be in the format mm:ss or m:ss and from 0:00 to 25:00	User:Validation Error, please contact admin. Admin:[Format,Fights,EndTime, FightID={FightID}]
Fights	Title	Length	Must be up to 60 characters	User:Validation Error, please contact admin. Admin:[Lookup,Fights>Title, FightID={FightID}]
FighterFights	FighterFightsID	Presence	FighterfightsID cannot be left empty	User:Validation Error, please contact admin. Admin:[Presence,FighterFights,FighterFightsID, FightID={FightID}]
FighterFights	FighterID	Lookup	Fighter must exist	User:Validation Error, please contact admin. Admin:[Lookup,FighterFights,FighterID, FighterFightsID={FighterFightsID}]
FighterFights	FightID	Lookup	Fight must exist	User:Validation Error, please contact admin. Admin:[Lookup,FighterFights>FightID, FighterFightsID={FighterFightsID}]

FighterFights	Corner	Lookup	(Normalised into lowercase) Corner must be either Red/A or Blue/B)	User:Validation Error, please contact admin. Admin:[Lookup,FighterFights,Corner, FighterFightsID={FighterFightsID}]
FighterFights	Result	Lookup	Must be either W/D/L/NC	User:Validation Error, please contact admin. Admin:[Lookup,FighterFights,Result, FighterFightsID={FighterFightsID}]
Fighters	FighterID	Presence	FighterID can't be left blank	User:Validation Error, please contact admin. Admin:[Presence,Fighters,FighterID, Name={Name}]
Fighters	Name	Length	Name can't be greater than 60 characters	User:Validation Error, please contact admin. Admin:[Length,Fighters,Name, FighterID={FighterID}]
Fighters	WeightClass	Lookup	(Will be normalised into lowercase) must follow the weight classes of sanctioned MMA. Being Atomweight,Strawweight,Flyweight,Bantamweight,Featherweight,Lightweight,Welterweight,Middleweight,Light Heavyweight,Heavyweight,Super Heavyweight, Catchweight. All these weight classes also can have the prefix of Women's for WMMA	User:Validation Error, please contact admin. Admin:[Lookup,Fighters,WeightClass, FighterID={FighterID}]
Fighters	Birthdate	Format and range	Must be in date format dd/mm/yyyy and can't be in the future	User:Validation Error, please contact admin. Admin:[Format,Fighters,Birthdate, FighterID={FighterID}]
Fighters	Gym	Length	Can't be greater than 80 characters	User:Validation Error, please contact admin. Admin:[Length,Fighters,Gym, FighterID={FighterID}]
Fighters	ELORating	Presence and range	Must be a float and in a sensible range being minimum 0 and maximum 3000	User:Validation Error, please contact admin. Admin:[Presence/Range,Fighters,EloRating, FighterID={FighterID}]
Fighters	Volatility	Presence and range	Must be a float and in a sensible range being minimum 0 and maximum 1	User:Validation Error, please contact admin. Admin:[Presence/Range,Fighters,Volatility, FighterID={FighterID}]
Fighters	Rating Deviation	Presence and range	Must be a float and in a sensible range being minimum 0 and maximum 350	User:Validation Error, please contact admin. Admin:[Presence/Range,Fighters,Rating Deviation, FighterID={FighterID}]
Fighters	ImageURL	Length and format check	Must be up to 255 characters	User:Validation Error, please contact admin. Admin:[Length/Format,Fighters,ImageURL, FighterID={FighterID}]

Exception handling/ python validation

Sub-Problem	Operation	Description	Error Message
1 during updates (7-11)	Asserts wrapped in Try	When doing the calculations the possible boundary will be asserted to make sure nothing goes wrong. If there is a mathematical error an exception will be raised and then displayed on a dialog box.	{Quality} calculation error. For example, if its ELO it will be "ELO calculation error"
2	Connection and execution wrapped in a try	When doing the SQL connections and executing a query it will be wrapped in a try statement to make sure that if the server fails the whole program doesn't break	Outputs SQL request exception in dialog box
6.1	HTTP request to get image from URL wrapped in a try statement	When sending the http request to get the image data it will be put into a try statement to stop the program from breaking if the request fails	Outputs http request exception in dialog box
19	General exceptions caught by sys	When there is an error it will go through an sys excepthook for it to be caught before it has any proper impact	Outputs error in a dialog box and also logs the traceback of the error for debugging
General	Nonetype catching	When the user does an input to avoid the program breaking if the result doesn't exist the program will first check if it exists	The program will deal with the subroutine sensibly most probably either replacing the value or returning the subroutine

Data integrity insurance

These features I have make sure the data is correct.

<u>Technique</u>	<u>Description</u>	<u>Example</u>
Data duplication flagging	The system will flag data that is most probably duplicated	The system will flag fights that involve the same fighters on the same dates
Statistical anomaly flagging	The system will flag features in the data that can't possibly happen	The system will flag a fighter who has fought equal to or more than 12 times in a year

Algorithms

Subproblem	Algorithm
1	<pre> assumes pi and e are known SUBROUTINE CalcElo(FighterA,FighterB,Fight) do Q=ln(10)/400 Fighter=(“SELECT * FROM Fighters WHERE FighterID="+FighterA+”;”) global beforeElo=Fighter[Elo] global beforeRD=Fighter[ratingDeviation] global beforeVolatility=Fighter[volatility] FightersOp=(“SELECT * FROM Fighters WHERE FighterID="+FighterB+”;”) opponentElo=(FightersOp[Elo]-1500)/173.7178 opponentRD=FightersOp[ratingDeviation]/173.7178 opponentVolatility=FightersOp[volatility] internalElo=(beforeElo-1500)/173.7178 internalRD=(beforeRD-1500)/173.7178 fighterFoughtIn=SQL(“SELECT * FROM Fights WHERE FighterFights.FightID=Fights.FightID AND FighterFights.FightID="+FighterA+”;”) lastFightID=((fighterFoughtIn.index(fighterFoughtIn-Fight))-1) lastFightDate=SQL(“SELECT Date from Events WHERE FightID="+lastFightID+”;”) FightDate=SQL(“SELECT Date from Events WHERE FightID="+Fight+”;”) inactivityTime=dateChangeDays(FightDate-lastFightDate) inactivityTime=inactivityTime/30 internalRD=((internalRD*2)+(beforeVolatility*2)+inactivityTime)^0.5 impact=(1+(3*(Q^2)*opponentElo)/(pi^2)))^0.5 expectedScore=(1+(e*(-impact*(internalElo-opponentElo))))^1 variance=((impact^2)*expectedScore*(1-expectedScore))^1 results=SQL(“SELECT Result FROM FighterFights WHERE fighterID="+FighterA+” AND fightID="+Fight+”;”) if results="W" result=1 elif results="L" --- else do result=0.5 end ratingImprovement=variance*(impact*(result-expectedScore)) beforeVolatilitySquared=beforeVolatility^2 ratingDeviationsSquared=ratingDeviation^2 searchTolerance=0.000001 SUBROUTINE volatilityFunction(x) do term1Numerator=(e^x)*((ratingImprovement^2)-(internalRD^2)-variance-(e^x)) term1Denominator=2*(((internalRD^2)+variance+(e^x)^2) term2Numerator=-x*ln(beforeVolatilitySquared) term2Denominator=0.5^2 term1=term1Numerator/term1Denominator term2=term2Numerator/term2Denominator return term1-term2 end lowerBoundVol=ln(beforeVolatilitySquared) higherBoundVol=lowerBoundVol while volatilityFunction(higherBoundVol)<0 do higherBoundVol=higherBoundVol+0.5 end while higherBoundVol-lowerBoundVol > searchTolerance do midpoint=(lowerBoundVol+higherBoundVol)/2 volMidpoint=volatilityFunction(midpoint) volLowerBound=volatilityFunction(volLowerBound) if volMidpoint*volLowerBound<0 do higherBoundVol=midpoint end else do lowerBoundVol=midpoint end end newVolatility=e^(lowerBoundVol/2) interimRatingDeviation=((beforeRD^2)+(newVolatility^2)^0.5 newRatingDeviation=((interimRatingDeviation^2)+(variance^1))^0.5 newElo=beforeElo+(newRatingDeviation*(impact*(result-expectedScore))) RAISE EXCEPTION IF Elo<0 OR Elo>3000 "Elo calculation error" RAISE EXCEPTION IF newVolatility<0 OR newVolatility>1 "Volatility calculation error" RAISE EXCEPTION IF newRatingDeviation<0 OR newRatingDeviation>350 "Rating Deviation calculation error" Return newVolatility,newRatingDeviation,newElo ARRAY Fights=SQL(“SELECT Fights.FightID FROM Fights, Events INNER JOIN Events ON Fights.EventID = Events.EventID ORDER BY Events.EventDate Asc”) FOR i TERMS IN LENGTH OF Fights do FighterA=SQL(“SELECT FighterID FROM FighterFights WHERE FightID="+Fights[FightID,i]+”AND Corner=Red;”) FighterB=SQL(“SELECT FighterID FROM FighterFights WHERE FightID="+Fights[FightID,i]+”AND Corner=Blue;”) CalcElo(FighterA,FighterB,Fights[FightID,i]) SQL(“UPDATE Fighters SET ratingDeviation="+newRatingDeviation+", volatility="+newVolatility+", EloRating="+newElo+", WHERE FighterID="+FighterA+”;”) CalcElo(FighterB,FighterA,Fights[FightID,i]) SQL(“UPDATE Fighters SET ratingDeviation="+newRatingDeviation+", volatility="+newVolatility+", EloRating="+newElo+", WHERE FighterID="+FighterA+”;”) end </pre>

2	<pre>SUBROUTINE sql(query) try and do connectionSettings= ("Driver=SQL SERVER SERVER=SERVER DATABASE=DATABASE Trusted_Connection=yes;") connection = pyodbc.connect(connectionSettings) if connection is successful do OUTPUT "Connection successful" connection.execute(query) end return connection.result() on Exception do OUTPUT error after do if connection is successful do connection.close() OUTPUT "Connection closed" end end end SUBROUTINE makeTable(query,table,headers) rows=sql(query) model=Qt.itemModel model.setHeaders(headers) for each row in rows do item=[Qt.item(field for each field in the row)] model.addRow(item) end table.setModel(model) table.setHeaders(headers) end</pre>
3	<pre>def clickHeader(header,type,request,toGet,table,headers) if type="qual" do query="SELECT "+toGet+" FROM Fighters WHERE "+Header+"="+request+";" else do query="SELECT "+toGet+" FROM Fighters ORDER BY "+request+";" end makeTable(query,table,headers) end</pre>

4

```
SUBROUTINE editDistance(string1,string2) do
    length1,length2=length(string1),length(string2)
    //swap if longer
    if length1 > length2 do
        string1,string2=string2,string1
        length1,length2=length2,length1
    end
    row=list(range(length1+1))
    for i terms from 1 to (length2+1) do
        lastRow,currentRow=currentRow,currentRow[i]+currentRow[0]
        for j terms from 1 to (length1+1) do
            add=lastRow[j]+1
            delete=currentRow[j-1]+1
            substitute=lastRow[j-1]
            change=substitute
            if string1[j-1] != string2[i-1] do
                change=change+1
            end
            currentRow[j]=minimum(add,delete,change)
        end
    end
    return currentRow[length1]
end

SUBROUTINE distanceOffTable(searchTerm) do
    rows=SQL("SELECT FighterID,Name FROM Fighters;")
    distances=[]
    for i terms from 1 to length(rows) do
        current=rows[i][1]
        self.editDistance(current,searchTerm)
        distances.append(rows[i][0],distance)
    end
    SORT distances by [1] ascending
    return distances
end
```

5/6 ProfileID is a global	<pre> leaderboard.onDoubleClick(//This is subproblem 5 profileID=index[0] fighterProfileInit(profileID)) SUBROUTINE fighterProfileInit(profileID) do fighterStats=SQL("SELECT * FROM Fighters WHERE FighterID="+ProfileID+";") try to do response=httpGet(fighterStats[8]) if "image" isn't in response do OUTPUT "NOT AN IMAGE" else do image=response.content fighterProfileImage.loadImage(image) end end On Exception do OUTPUT Exception end pastFightsQuery=" SELECT Fighters.Name AS opponentName Events.Name AS eventName Fights.Method,Fights.EndRound,Fights.EndTime FROM FighterFights,Fights,Events INNER JOIN Fighters ON FighterFights.OpponentID = Fighters.FighterID INNER JOIN Fights ON FighterFights.FightID = Fights.FightID INNER JOIN Events ON Fights.EventID = Events.EventID WHERE FighterFights.FighterID = "+ProfileID+" ORDER BY Events.Date DESC; " makeTable(pastFightsQuery,pastFightsTable,["Opponent","Event","Result","Round","Time"]) totalWins=sql("SELECT SUM(CASE WHEN Result = 'W' THEN 1 ELSE 0 END) AS totalWins FROM FighterFights WHERE FighterID="+ProfileID+";") totalLosses=sql("SELECT SUM(CASE WHEN Result = 'L' THEN 1 ELSE 0 END) AS totalWins FROM FighterFights WHERE FighterID="+ProfileID+";") totalDraws=sql("SELECT SUM(CASE WHEN Result = 'D' OR Result = 'NC' THEN 1 ELSE 0 END) AS totalWins FROM FighterFights WHERE FighterID="+ProfileID+";") pastFights=SQL(pastFightsQuery) resultChanged=False winStreak=False lossStreak=False inc=1 while resultChanged = False do if pastFights[result,inc] = pastFights[result,inc-1] do if pastFights[result,inc] = "W" do winStreak=True inc=inc+1 end else do lossStreak=True inc=inc+1 end end else do resultChanged=True end end totalSubs=SQL("SELECT COUNT(Fights.FightID) FROM FighterFights,Fights WHERE Fights.Result='Win' AND FighterFights.Result='SUB' AND FighterFights.FightID=Fights.FightID AND FighterFights.FighterID="+ProfileID+";") totalKos=SQL("SELECT COUNT(Fights.FightID) FROM FighterFights,Fights WHERE Fights.Result='Win' AND FighterFights.Result='KO' AND FighterFights.FightID=Fights.FightID AND FighterFights.FighterID="+ProfileID+";") totalDec=SQL("SELECT COUNT(Fights.FightID) FROM FighterFights,Fights WHERE Fights.Result='Win' AND FighterFights.Result='DEC' AND FighterFights.FightID=Fights.FightID AND FighterFights.FighterID="+ProfileID+";") totalFights=totalWins+totalDraws+totalLosses winRate=(100/totalFights)*totalWins finishRate=(100/totalFights)*(totalSubs+totalKos) end </pre>
8	<pre> def top10Opponents(fighterID) do opponentsList=SQL("SELECT Name FROM Fighters WHERE FighterID=(SELECT FighterID FROM FighterFights WHERE FighterFights.FightID=(SELECT FighterID From FighterFights WHERE FightID=(SELECT FightID from FighterFights WHERE FighterID="+fighterID+") AND NOT FighterID="+fighterID+";") fighterWeight=SQL("SELECT WeightFROM Fighters WHERE FighterID="+FighterID+";") top10division=SQL("SELECT Name FROM Fighters WHERE Weight="+FighterWeight+"; ORDER BY EloRating LIMIT 9") top10OpponentsList=[] for x terms from 0 to 9 do for y terms for length(opponentsList) do if top10division[x]=opponentsList[y] do top10OpponentsList.append(opponentsList[y]) end end end end end </pre>

9

```
global oddsComboCurrentWeightClass
SUBROUTINE initialiseBoxes() do
    currentA=fighterABox.data
    currentB=fighterABox.data
    fighterABox.Clear
    fighterBBox.Clear
    if oddsComboCurrentWeightClass do
        query="SELECT FighterID, Name FROM Fighters WHERE WeightClass="+oddsComboCurrentWeightClass
    else do
        query="SELECT FighterID,Name FROM Fighters;"
    end
    fighterList=SQL(query)
    currents=[currentA,currentB]
    comboBoxes=[fighterABox,fighterBBox]
    if fighterList does not exist do
        fighterList=[]
    end
    for i terms in range of 1 to 2 do
        for fighterID,fighterName in fighterList do
            comboBoxes[i].addItem(fighterName,fighterID)
            index = comboBoxes[i].findData(currents[i])
            if index != -1 do
                comboBoxes[i].setIndex(index)
            end
        end
    end
end

SUBROUTINE comboBoxesChanged(index,box) do
    if box = "A" do
        currentID = fighterABox.data
    end
    elif box = "B" do
        currentID = fighterBBox.data
    end
    if currentID exists do
        currentFighter = SQL("SELECT * FROM Fighters WHERE FighterID="+currentID+";")
        if currentFighter exists do
            oddsComboCurrentWeightClass=currentFighter[2]
        end
        else do
            oddsComboCurrentWeightClass= None
        end
    end
    else do
        oddsComboCurrentWeightClass=None
    end
    initialiseBoxes()
    currentBID = fighterBBox.data
    if fighterAID exists do
        fighterAdata=SQL("SELECT * FROM Fighters WHERE FighterID="+currentID+";")
        fighterAurl= fighterAdata[8] if fighterAdata and length(fighterAdata) > 8 else None
        loadImage(fighterAurl,fighterAoddsImage)
    end
    if fighterBID exists do
        fighterBdata=SQL("SELECT * FROM Fighters WHERE FighterID="+currentBID+";")
        fighterBurl= fighterBdata[8] if fighterBdata and length(fighterBdata) > 8 else None
        loadImage(fighterBurl,fighterBoddsImage)
    end
end
```

10.1	<pre> #10.1 Username=input() Password=input() signUpButton.onClick(Result = SQL("SELECT 1 FROM dbo.Users WHERE Username = "+Username+";") If Result do OUTPUT "Username already exists" Return end salt = randomBytes(16) hash = hash(pbkdf2_hmac,function=sha256,toEncode=Password,salt=salt,Iterations=100000) base64salt = base64encode(salt) base64hash = base64encode(hash) hashedPassword = base64salt+base64hash SQL("INSERT INTO Users (Username, HashedPassword,SubOpinion,KoOpinion,DecisionOpinion,isAdmin) VALUES ("+Username+", "+Password+",1,1,1,0);") sessionUsername = Username LOGIN()) </pre>
10.2	<pre> #10.2 storedAccount = SQL("SELECT * FROM dbo.Users WHERE Username = "+Username+";") Username=input() Password=input() if storedAccount do storedPassword=storedAccount[0] base64salt and base64hash = split(storedPassword AT ":") decodedHash = base64decode(base64hash) decodedSalt = base64decode(base64salt) newhash = hash = hash(pbkdf2_hmac,function=sha256,toEncode=Password,salt=decodedSalt,Iterations=100000) if decodedHash = newhash do LOGIN() end if decoded != newhash do OUTPUT "Login unsuccessful" end end </pre>
10.3	<pre> #10.3 On LOGIN() do sessionID=SQL("SELECT userID FROM dbo.Users WHERE Username = "+Username+";") end </pre>
10.4	<pre> #10.4 On LOGIN() do isAdmin=SQL("SELECT isAdmin FROM dbo.Users WHERE Username = "+Username+";") end </pre>
10.5	<pre> #10.5 KoOpinion=0 SubOpinion=0 DecisionOpinion=0 koStars=[star1,star2,star3,star3,star4] subStars=[star1,star2,star3,star3,star4] decStars=[star1,star2,star3,star3,star4] submitSurveyButton.onClick(for i FROM 1 to koStars do if i.beenClicked=True KoOpinion=i return end for i FROM 1 to subStars do if i.beenClicked=True SubOpinion=i return end for i FROM 1 to decStars do if i.beenClicked=True DecisionOpinion=i return end SQL("UPDATE dbo.Users SET SubOpinion = "+SubOpinion+", KoOpinion = "+KoOpinion+", DecisionOpinion = "+DecisionOpinion+" WHERE Username="+Username+";")) </pre>

11	<pre> SUBROUTINE addFighter do SQL("INSERT INTO Fighters (Name, WeightClass, Birthdate, Gym, EloRating, Volatility, RatingDeviation) VALUES ('John Smith','Lightweight','1987-01-01','gym',1500,350,200)") end </pre>
12	<pre> SUBROUTINE scrapeEvents(startDate) do finished=False event=startDate events=[] While finished=False do eventdata=ufcScrape.getEventData(startDate) if eventdata exists do events.append[] events[ID,event]=eventdata[ID] events[name,event]=eventdata[name] events[city,event]=eventdata[city] events[date,event]=eventdata[date] event=event+1 end if eventdata does not exist do finished=True end end return events end SUBROUTINE ufcScrape(startDate) do events=scrapeEvents(startDate) for i terms in length of events do SQL("INSERT INTO Events (name,date,location) VALUES "events[0],events[1],events[2],events[3];") fightData=ufcScrape.getFightData(eventName=events[0]) for x terms in length of fightData do //get fight data fights.append[] fighterFights.append[] fights[fightID,x]=fightData[fightID,x] fights[eventID,x]=events[i] fights[Method,x]=fightData[result,x] fights[endRound,x]=fightData[endRound,x] fights[endTime,x]=fightData[endTime,x] if fightData[titleFight,x]=True do fights[titleFight,x]=fightData[weightclass,x] end sql("INSERT INTO Fights (FightID,EventID,Method,EndRound,EndTime,Title) VALUES ("fights[fightID,x]+fights[eventID,x]+fights[Method,x]+fights[endRound,x]+fights[endTime,x]+fights[titleFight,x];") //get fighterfights data fighterA=fightData[fighter1,x] fighterB=fightData[fighter2,x] if fightdata[winner,x]=fighterA do sql("INSERT INTO FighterFights (FighterID,FightID,Corner,Result) VALUES " fighterA,fights[fightID,x],Red,Win;") sql("INSERT INTO FighterFights (FighterID,FightID,Corner,Result) VALUES " fighterB,fights[fightID,x],Blue,Loss;") end if fightdata[winner,x]=fighterB do sql("INSERT INTO FighterFights (FighterID,FightID,Corner,Result) VALUES " fighterA,fights[fightID,x],Red,Loss;") sql("INSERT INTO FighterFights (FighterID,FightID,Corner,Result) VALUES " fighterB,fights[fightID,x],Blue,Win;") end //get fighter data fighterData=getFightData(FighterID=FighterA) sql("INSERT INTO fighters (FighterID,Name,WeightClass,Birthdate,Gym,EloRating,Volatility,RatingDeviation) VALUES "fighterData[name]+fighterData[weightclass]+fighterData[Birthdate]+fighterData[Gym]+1500+300+450;") fighterData=getFightData(FighterID=FighterB) sql("INSERT INTO fighters (FighterID,Name,WeightClass,Birthdate,Gym,EloRating,Volatility,RatingDeviation) VALUES "fighterData[name]+fighterData[weightclass]+fighterData[Birthdate]+fighterData[Gym]+1500+300+450;") end end end </pre>

13
(Should
be done
on any ui
input
subroutin
e)

```
//dataErrors is a global table
SUBROUTINE findDuplicates do
    fighterdups=SQL("
    SELECT FighterID,Name,Birthdate
    FROM Fighters
    GROUP BY Name,Birthdate
    HAVING COUNT(*)>1")
    if fighterDups exists do
        for i terms in length of fighterDups do
            dataErrors.append(["Duplicate","Fights","Name/Birthdate","FighterID"])
        end
    end
    eventsdups=SQL("
    SELECT EventID,Name,Date
    FROM Events
    GROUP BY Name,Date
    HAVING COUNT(*)>1")
    if eventsdups exists do
        for i terms in length of eventsdups do
            dataErrors.append(["Duplicate","Events","Name/Date","FighterID"])
        end
    end
    fightdups=SQL("
    SELECT FighterFightID,FighterID,FightID
    FROM FighterFights
    GROUP BY FighterID,FightID
    HAVING COUNT(*)>1")
    if fightdups exists do
        for i terms in length of fightdups do
            dataErrors.append(["Duplicate","Events","FightID/FighterID","eventsID"])
        end
    end
    beltdups=SQL("
    SELECT beltID,weight,fighterID
    FROM belts
    GROUP BY weight,fighterID
    HAVING COUNT(*)>1")
    if beltdups exists do
        for i terms in length of beltdups do
            dataErrors.append(["Duplicate","Belts","weight/fighterID","beltID"])
        end
    end
end

SUBROUTINE lookupCheck(data,lookup) do
    valid=False
    for x terms in length of lookup do
        if lookup[x]=data do
            valid=True
        end
    end
    return valid
end

SUBROUTINE rangeCheck(data,lowerBound,upperBound) do
    valid=False
    if lowerBound<data do
        if upperBound>data do
            valid=True
        end
    end
    return valid
end

SUBROUTINE lengthCheck(data,max) do
    valid=False
    if data < max do
        valid=True
    end
    return valid
end

SUBROUTINE presenceCheck(data) do
    valid=False
    if data exists do
        valid=True
    end
    return valid
end
```

14	<pre> SUBROUTINE userApproval(userID,fighterID,operation) do checkDuplicate=SQL("SELECT * FROM Approvals WHERE userID="+userID+"AND fighterID="+fighterID+" AND Rating="+operation+";") if checkDuplicate exists do return end checkExists=SQL("SELECT ApprovalID FROM Approvals WHERE userID="+userID+"AND fighterID="+fighterID+";") if checkExists exists do SQL("DELETE FROM Approvals WHERE ApprovalID="+ApprovalID+";") end SQL("INSERT INTO Approvals(userID,FighterID,Rating) VALUES (" +userID+", "+fighterID+", "+operation+")") end SUBROUTINE approvalRating(fighterID) do totalRatings=SQL("SELECT COUNT(*) FROM Approvals WHERE FighterID="+FighterID+";") totalUpRatings=totalRatings=SQL("SELECT COUNT(*) FROM Approvals WHERE FighterID="+FighterID+" AND Rating=True;") percentageRating=(totalUpRatings/totalRatings) return percentageRating end //ui will do userApproval and then approvalRating to update </pre>
15	<pre> SUBROUTINE entertainmentScore(fighterID,userID) do averages=[0,0,0] fighterAverages=[0,0,0] entertainmentScore=100 totalFights=SQL("SELECT COUNT(*) FROM Fights;") totalSubs=SQL("SELECT COUNT(*) FROM Fights WHERE Method=SUB;") totalDecs=SQL("SELECT COUNT(*) FROM Fights WHERE Method=DEC;") totalKos=SQL("SELECT COUNT(*) FROM Fights WHERE Method=KO;") averages[0]=(totalSubs/totalFights)*100 averages[1]=(totalDecs/totalFights)*100 averages[2]=(totalKos/totalFights)*100 fighterTotalFights=SQL("SELECT COUNT(*) FROM Fights WHERE Fights.FightID=FighterFights.FightID AND FighterFights.FighterID="+fighterID+";") fighterTotalSubs=SQL("SELECT COUNT(*) FROM Fights WHERE Method=SUB AND Fights.FightID=FighterFights.FightID AND FighterFights.FighterID="+fighterID+";") fighterTotalDecs=SQL("SELECT COUNT(*) FROM Fights WHERE Method=DEC AND Fights.FightID=FighterFights.FightID AND FighterFights.FighterID="+fighterID+";") fighterTotalKos=SQL("SELECT COUNT(*) FROM Fights WHERE Method=KO AND Fights.FightID=FighterFights.FightID AND FighterFights.FighterID="+fighterID+";") fighterAverages[0]=(fighterTotalSubs/fighterTotalFights)*100 fighterAverages[1]=(fighterTotalDecs/fighterTotalFights)*100 fighterAverages[2]=(fighterTotalKos/fighterTotalFights)*100 userOpinions=SQL("SELECT SubOpinion,DecisionOpinion,KoOpinion FROM Users WHERE UserID="+UserID+"") for i terms in range 0 to 2 do if fighterAverages[i]>averages[i] do entertainmentScore=entertainmentScore-(8.325*(5-userOpinions[i])) end end return entertainmentScore end SUBROUTINE fightEntertainmentScore(fighterAID,fighterBID,userID) do return (entertainmentScore(fighterAID,userID),entertainmentScore(fighterBID,userID))/2 end </pre>
16	<pre> SUBROUTINE anomalyCheck do impossibleFights=SQL("SELECT FightID FROM Fights WHERE EXTRACT(SECONDS FROM EndTime)<=1 AND EXTRACT(MINUTES FROM EndTime)=0;") if impossibleFights exists do dataErrors.append(["Anomaly","Fights",FightID]) end minDate=SQL("SELECT MIN(EXTRACT(YEAR FROM Date)) FROM Events;") maxDate=SQL("SELECT MAX(EXTRACT(YEAR FROM Date)) FROM Events;") fighterIdList=SQL("SELECT FighterID FROM Fighters;") for year terms in range of minDate to maxDate do for fighter terms in range of length(fighterIdList) do fightCount=("SELECT COUNT(*) FROM FighterFights WHERE FighterID="+fighterIdList[fighter]+" AND FighterFights.FightID=Fights.FightID AND Fights.EventID=Events.EventID AND Events.EXTRACT(YEAR FROM Date)="+year+";") if fightCount >= 12 do dataErrors.append(["Anomaly","FighterFights",fighterIdList]) end end end end </pre>

17.1	<pre> #17.1 function navigate(toNavigate) { page is global pages=[fighterProfiles, leaderboard, login, modifyFighter, modifyFighterList, oddsPredictor, survey] for i FROM 1 to length(pages) do hidePage(pages[i]) end pages[toNavigate].show page=pages[toNavigate] } </pre>
17.2	<pre> #17.2 //Should be present in some way or form in every user interaction element if page = specificPage do function() end //Or it could be present as if page != specificPage do return end </pre>
18	<pre> SUBROUTINE openBeltsPage do makeTable("SELECT * FROM Belts",beltsTable,"BeltID,Weight/Name,FighterID") end SUBROUTINE addBelt do SQL("INSERT INTO Belts(Weight/Name,FighterID) VALUES (Blank,NULL)") makeTable("SELECT * FROM Belts",beltsTable,"BeltID,Weight/Name,FighterID") end SUBROUTINE selectedBelt(index) do global currentBeltTableIndex=index end SUBROUTINE deleteBelt do global currentBeltTableIndex beltID=currentBeltTableIndex[beltID] SQL("DELETE FROM Belts WHERE beltID="+beltID+";") titlesToRetire=SQL("SELECT FightID from Fights where Title="+currentBeltTableIndex[weight]+";") for i terms in length(timeToRetire) do SQL("UPDATE Fights SET Title='RETIRED' WHERE fightID="+titlesToRetire[i]+";") end makeTable("SELECT * FROM Belts",beltsTable,"BeltID,Weight/Name,FighterID") end //subroutine done on text edit SUBROUTINE editBeltHolder do wait 20 SECONDS global selectedBelt SQL("UPDATE Belts Set FighterID="+getTableText(currentBeltTableIndex[weight])+ " WHERE beltID="+currentBeltTableI end </pre>

20	<pre> SUBROUTINE autoBeltAssignment() do unassignedBelts=SQL("SELECT BeltID FROM Belts WHERE Weight=NULL;") for i terms in length(unassignedBelts) do SQL("UPDATE Belts SET FighterID=(SELECT FighterID FROM FighterFights WHERE FightID=(SELECT FightID FROM FIGHTS WHERE Title="+unassignedBelts[i]+" ORDER BY Date DESC LIMIT 1)) WHERE BeltID="+unassignedBelts[i]+";") end end SUBROUTINE autoBeltCreation() do beltLessDivisions=[] weightClasses=SQL("SELECT DISTINCT Title FROM Fights") for i terms in length(weightClasses) do checkForBelt=SQL("SELECT * FROM Belts WHERE Weight="+weightClasses[i]+";") if checkForBelt does not exist do beltLessDivisions.append(weightClasses[i]) end end for i terms in length(beltLessDivisions) do SQL("INSERT INTO Belts (Weight,FighterID) VALUES ("+beltLessDivisions[i]+",NULL);") autoBeltAssignment() end end </pre>
21.1	<pre> #21 viewAsUserButton.onClick do navigate(leaderboard) end </pre>
21.2	<pre> #21.2 if isAdmin = True do resize(oddsCheckerButton,length=50%) move(oddsCheckerButton,x=50%) show(manageListButton) end </pre>